FreeRTOS

一、课程目标

1. 理解操作系统基本原理
2. 移植FreeRTOS
3. 熟悉在FreeRTOS中创建和管理任务
4. 了解FreeRTOS的常用功能，如消息队列、信号量、定时器、中断等

二、课程引入

1、系统

PC端操作系统： Windows， Linux，MacOS

移动端：ios , 鸿蒙，Android(基于Linux)

服务器端：Linux, Windows Server

（单片机）嵌入式： RTX，RT-Thread，FreeRTOS, ucOS，Linux

2、为什么要使用操作系统?

如果我们要实现一个功能：两个LED灯，一个LED亮3秒，灭1秒，另一个LED亮2，灭秒5。

下面是其伪代码，能看出来有什么问题吗?

```
LED1_Blink:

     LED1_ON

     Delay(3)

     LED1_OFF

     Delay(1)

LED2_Blink:

     LED2_ON

     Delay(2)

     LED2_OFF

     Delay(5)

main:
```

```
while:
  LED1_Blink()
  LED2_Blink()
```

1、裸机开发

裸机开发指的是在没有操作系统（OS）或者其他高级软件支持的情况下，直接在裸机硬件上进行软件开发的过程。在裸机开发中，开发者需要直接面对硬件层面的操作和控制，亲自管理CPU、内存以及I/O资源，而不依赖于任何操作系统提供的抽象层或者服务。

我们前面学习的STM32单片机代码都属于 裸机开发 。

1、嵌入式系统要不要使用操作系统?

- 不使用操作系统：
  1. 任务单一
  2. 希望系统非常省电
- 使用操作系统：
  1. 同时执行多个任务
  2. 对功耗没有要求
  3. 希望可以有一个程序自动帮我们管理内存
- 嵌入式操作系统有什么要求?
  1. 占用ROM足够小
  2. 运行内存要足够小
  3. 一般没有界面

2、裸机运行和操作系统运行的区别

功能	裸机	操作系统
任务数量	适合运行单任务	适合运行多任务
内存管理	只能手动处理	可以使用操作系统的内存管理功能，提高开发和运行效率
其它功能	都需要自己写	可以使用操作系统提供的强大功能

3、FreeRTOS（Real Time Operation System）

FreeRTOS 是市场领先的面向微控制器和小型微处理器的实时操作系统 (RTOS)，与世界领先的芯片公司合作开发，现在每 170 秒下载一次。FreeRTOS 通过 MIT 开源许可免费分发，包括一个内核和一组不断丰富的 IoT 库，适用于所有行业领域。FreeRTOS 的构建突出可靠性和易用性。

Free即免费的，RTOS的全称是Real time operating system,中文就是实时操作系统。注意：RTOS不是指某一个确定的系统，而是指一类操作系统。比如：uc/OS,FreeRTOS，RTX，RT-Thread等这些都是RTOS类操作系统。

操作系统允许多个任务同时运行，这个叫做多任务。实际上，一个处理器核心在某一时刻只能运行一个任务。操作系统中任务调度器的责任就是决定在某一时刻究竟运行的哪个任务。任务调度在各个任务之间的切换非常快，就给人们造成了同一时刻有多个任务同时运行的错觉。

某些操作系统给每个任务分配同样的运行时间，时间到了就轮到下一个任务，比如Unix操作系统，FreeRTOS操作系统则是由用户给每个任务分配一个任务优先级，任务调度器就可以根据此优先级来决定下一刻应该运行哪个任务。

FreeRTOS是RTOS系统中的一种，FreeRTOS十分的小巧，可以在资源有限的微控器中使用，当然FreeRTOS不仅仅局限于在微控器中使用，但从文件数中来说FreeRTOS要比uc/OS小的多。RL78 移植可在不到 4K 字节的 RAM 中创建 13 项任务、2 个队列和 4 个软件定时器！

4、为什么选择FreeRTOS?

中文官网：https://www.freertos.org/zh-cn-cmn-s

RTOS类系统有很多，比如说UC/OS，资料很多，尤其是中文资料，那为什么要选择FreeRTOS呢?

① FreeRTOS是开源免费的，学习RTOS操作系统的话，UC/OS是首先，但是做产品的话，免费的FreeRTOS操作系统就是个不错的选择。

② 很多半导体厂商产品的SDK（Software Development Kit）软件开发工具包，就使用FreeRTOS作为其操作系统，尤其是WIFI、蓝牙这些带有协议栈的芯片或模块。

③ 简单，因为FreeRTOS的文件数量很少。

5、FreeRTOS的特点

FreeRTOS是一个可裁剪的小型的RTOS系统，其特点包括：

① 任务调度：FreeRTOS的内核支持抢占式，协同式和时间片调度。

② 任务通信和同步：提供了队列、信号量等机制，支持任务之间的通信和同步，确保数据的安全传递。

③ 内存管理：提供简单的内存管理机制，适用于嵌入式环境，有效利用有限的内存资源。

④ 定时器和中断处理：支持定时器功能，能够处理中断，提供了可靠的实时性能。

⑤ 开发社区：拥有庞大的用户社区，开发者可以在社区中获取支持、解决问题，并分享经验。

⑥ 可移植性：设计注重可移植性，可以轻松地移植到不同的硬件平台，提高了代码的重用性。

6、FreeRTOS的发展

起源与早期发展（2003年）：

- Richard Barry 最早开发 FreeRTOS 的目的是为了给嵌入式系统提供一个轻量级的实时操作系统，使开发者能够在资源受限的设备上更好地实现多任务处理。
- FreeRTOS 从一开始就定位为开源项目，以 MIT 许可证发布，这为它在嵌入式开发社区中迅速获得了关注和支持。

逐渐普及与广泛应用（2005年-2017年）：

- FreeRTOS 的开放性和模块化设计，使其适应多种架构（ARM、MIPS、x86等），因此被许多半导体公司和嵌入式设备制造商广泛采用。

- 随着物联网（IoT）和智能设备的普及，FreeRTOS 成为众多微控制器（如 STM32、ESP32、PIC等）开发的首选实时操作系统。

AWS 的收购和 Amazon FreeRTOS（2017年）：

- 2017年，亚马逊 (Amazon) 宣布收购 FreeRTOS，并与 Richard Barry 合作进一步扩展 FreeRTOS，以更好地支持 IoT 设备的云端连接。
- 在 AWS 的支持下，FreeRTOS 得到增强，推出了 Amazon FreeRTOS 版本，增加了与 AWS 云服务的无缝集成功能，如 MQTT 协议、设备影子（Device Shadow）等。这样使得 FreeRTOS 能更方便地连接到 Amazon IoT Core。

持续更新与开源社区的贡献（2018年-至今）：

- Amazon FreeRTOS 逐步引入更多的安全性和网络协议支持（如 OTA、TLS 等），使其在工业物联网和消费级 IoT 应用中具有更强的竞争力。
- AWS 继续维护和更新 FreeRTOS，同时社区也积极贡献，使其在保持小巧、灵活的特点的同时，支持更多的硬件平台和应用需求

三、FreeRTOS基础知识

1、多任务处理

·**内核**是操作系统的核心组件。Linux 等通用操作系统采用的内核 允许多个用户看似同时访问计算机的处理器。这些用户可以各自执行多个程序， 看起来像是并发运行 。

每个执行的程序由操作系统控制下的一个或多个**线程**实现。如果操作系统能够以这种方式执行多个线程，则称为**多任务处理**。 像 FreeRTOS 这样的小型 RTOS 通常将线程称为 任务 ，因为它们不支持虚拟内存，因此进程和线程之间没有区别。

使用多任务操作系统可以简化原本复杂的软件应用程序的设计：

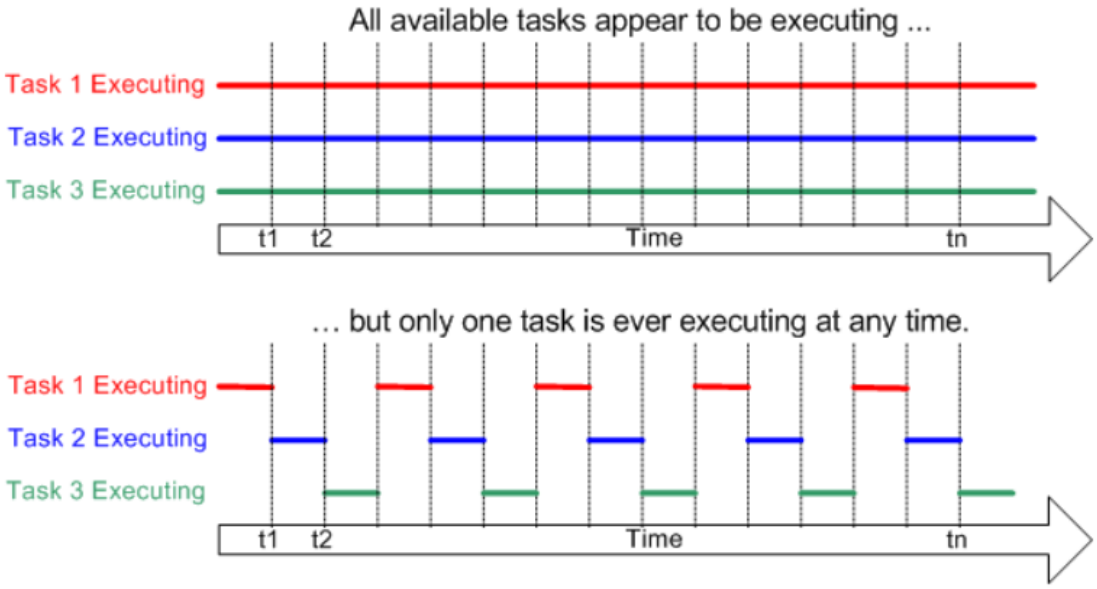
- 操作系统的多任务处理和任务间通信功能允许将复杂的应用程序 划分为一组更小且更易于管理的任务。
- 这种划分可以简化软件测试，确保团队分工明确，并促进代码复用。
- 复杂的时序和排序细节将由 RTOS 内核负责，从而减轻了应用程序代码的负担。

多任务操作系统可以快速切换任务， 使所有任务**看起来**像是同时在执行。下图展示了 三个任务相对于时间的执行模式

任务名称用不同颜色标示，并写在左侧。

时间从左向右移动， 彩色线条显示在特定时间执行的任务。

上方展示了所感知的并发执行模式， 下方展示了实际的多任务执行模式。



2、FreeRTOS的滴答 tick

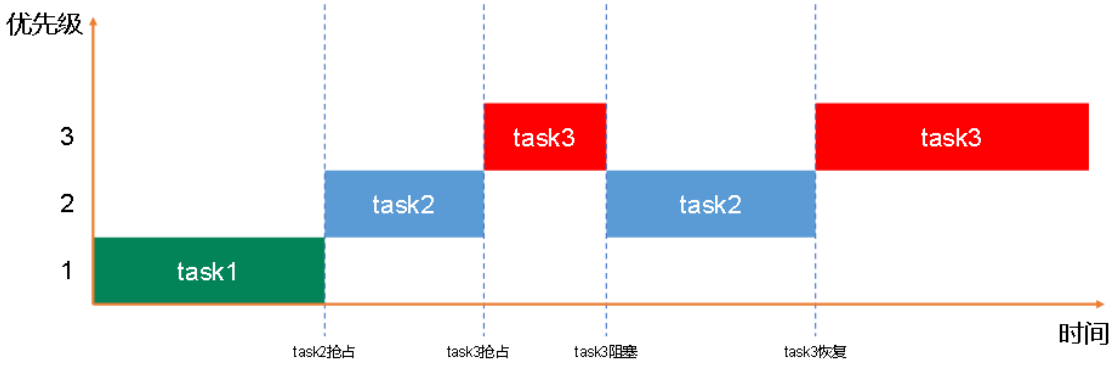
在 FreeRTOS 中，滴答（tick）是一个定时器中断信号，用于控制任务调度的时间基准。滴答是一个固定频率的中断，由系统定时器产生，通常以毫秒为单位（如 1 ms 或 10 ms），但具体频率可以在 FreeRTOS 配置中设置。每次滴答发生时，FreeRTOS 的内核会更新内部的时间计数器，这个计数器用于触发任务切换和延时处理

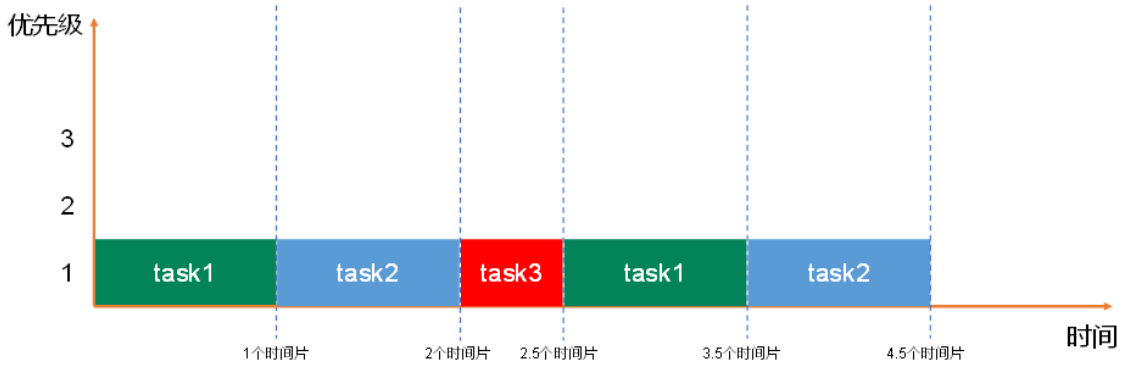
3、任务调度器

调度器是内核中负责决定在特定时间应执行什么任务的部分。内核 可以在任务的生命周期内多次暂停并恢复该任务。

FreeRTOS 默认使用**固定优先级**的 抢占式调度策略 ，对**同等优先级**的任务执行 时间片轮询 调度：

- “固定优先级”是指调度器不会永久更改任务的优先级， 但可能会因 优先级继承 而暂时提高任务的优先级。
- “抢占式”是指调度器始终运行优先级最高且可运行的 RTOS 任务。这确保了高优先级任务能够及时响应，并提高了系统的实时性。
- “轮询调度”是指具有相同优先级的任务轮流进入运行状态。
- “时间切片”是指调度器会在每个 tick 中断上在同等优先级任务之间进行切换， tick 中断之间的时间构成一个时间切片。（tick 中断是 RTOS 用来衡量时间的周期性中断。）
- 需要注意的是高优先级的任务并不会一直执行，导致低优先级的任务无法得到执行。如果高优先级任务**等待某个资源（延时或等待信号量等）**而无法执行，调度器会选择执行其他就绪的高优先级的任务





4、 上下文切换

在 FreeRTOS 中，“上下文切换”指的是在运行不同任务之间切换 CPU 执行状态的过程。使得多个任务可以在单个处理器上“并发”执行，尽管实际上它们是交替运行的。

①上下文切换的含义

在嵌入式操作系统中，每个任务有自己的**执行上下文**，包括以下内容：

- **CPU 寄存器状态**：当前任务的寄存器内容（如程序计数器、堆栈指针、通用寄存器等），它们定义了任务的执行位置和状态。
- **堆栈**：任务的堆栈保存了任务的局部变量、函数调用记录等信息，帮助恢复任务的执行流程。
- **其他状态信息**：如任务的优先级、任务状态（就绪、阻塞、挂起等）以及其他内核相关的信息。

上下文切换就是保存当前任务的执行上下文，并恢复下一个任务的执行上下文，以实现任务间的切换。

②上下文切换的触发条件

1. **滴答中断**：如上所述，滴答中断（tick interrupt）是 FreeRTOS 的时间基准。当滴答发生时，系统会检查是否需要切换到优先级更高的任务。
2. **任务延时或阻塞**：当任务调用延时函数（如 `vTaskDelay` ）或因等待资源而阻塞（如 `xQueueReceive` ），当前任务会被放入阻塞状态，调度器会切换到另一个就绪任务。
3. **任务优先级变更**：如果任务的优先级被动态调整，或有新任务被创建、就绪，且其优先级比当前任务高，FreeRTOS 会立即触发上下文切换，以优先执行优先级更高的任务。
4. **其他中断**：当其他中断（如外部硬件中断）发生时，如果该中断中唤醒了更高优先级的任务，也会触发上下文切换。

③上下文切换的步骤

1. **保存当前任务的上下文**：将当前任务的寄存器内容、堆栈指针等状态信息保存到任务控制块（TCB）中，以便在下次调度到该任务时可以恢复它。

2. **调度器选择新任务**：FreeRTOS 的调度器根据任务的优先级和状态（如就绪、阻塞）选择下一个要运行的任务。
3. **恢复新任务的上下文**：从新任务的任务控制块中加载寄存器内容、堆栈指针等信息，恢复新任务的执行状态。

④上下文切换的代价

上下文切换会占用一定的 CPU 时间，因为它需要保存和恢复任务的状态，这会带来性能开销，尤其在频繁切换时。因此，FreeRTOS 的设计中注重优化上下文切换的效率，例如通过 合理分配任务优先级 、设置适当的滴答频率 等来减少不必要的切换

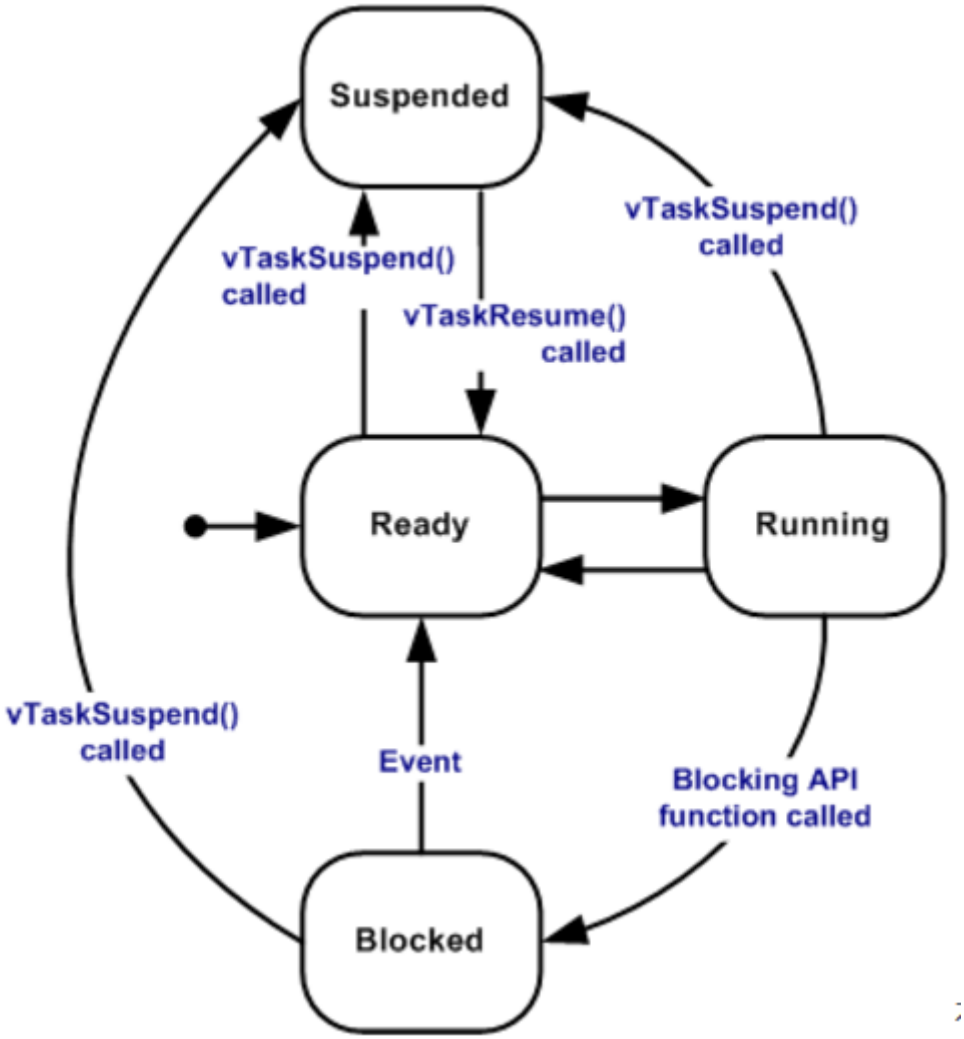
5、空闲任务

RTOS 调度器启动时，**自动创建空闲任务**，以确保始终存在一个能够运行的任务。

空闲以最低优先级创建，以确保如果有更高的优先级应用程序任务处于准备就绪状态，空闲任务则不使用任何 CPU 时间。

空闲任务负责释放被删除的任务的内存。

6、任务状态

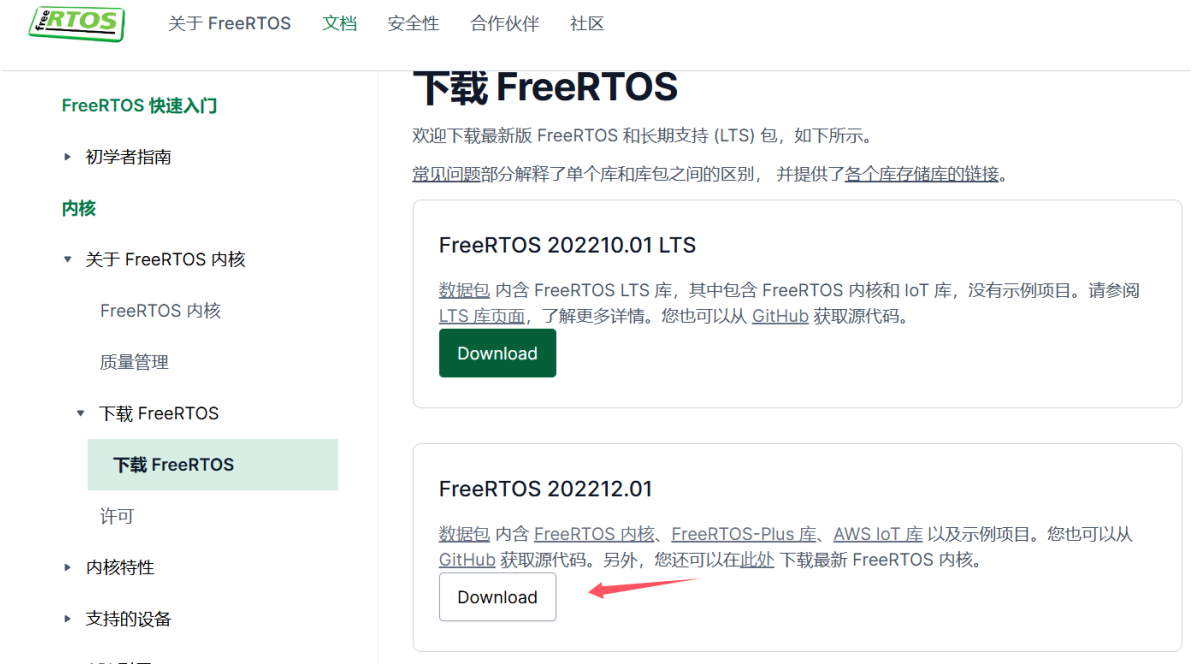


- 就绪（Ready）：准备就绪任务指那些能够执行（它们不处于阻塞或挂起状态），但目前没有执行的任务，因为同等或更高优先级的不同任务已经处于运行状态。

- 运行（Running）：该状态表明任务正在执行， 此时它占用处理器， FreeRTOS 调度器选择运行的永远是处于**最高优先级**的就绪态任务，当任务被运行的一刻，它的任务状态就变成了运行态。
- 阻塞（Blocked）： 如果任务当前正在 等待时间或外部事件 ，则该任务被认为处于阻塞状态。 例如，如果一个任务调用vTaskDelay()，它将被阻塞（被置于阻塞状态）， 直到延迟结束。 任务也可以通过阻塞来等待队列、信号量、事件组、通知或信号量 事件。处于阻塞状态的任务通常有一个"超时"期， 超时后任务将被超时，并被解除阻塞， 即使该任务所等待的事件没有发生。 “阻塞”状态下的任务不使用任何处理时间，不能 被选择进入运行状态。。
- 挂起态(Suspended)： 与“阻塞”状态下的任务一样， “挂起”状态下的任务不能 被选择进入运行状态，但处于挂起状态的任务 没有超时。相反，任务只有在分别通过 vTaskSuspend() 和 xTaskResume() API 调用明确命令时 才会进入或退出挂起状态。

四、FreeRTOS移植方式一

官方下载：https://www.freertos.org



移植步骤(以FreeRTOSv202212.00版本为例)：

1. 新建一个可以运行的项目
2. 在项目中添加存储FreeRTOS文件的文件夹，建议： FreeRTOS/src、 FreeRTOS/port、FreeRTOS/inc
3. **FreeRTOS的头文件**：复制\FreeRTOS\Source\include中的所有文件到FreeRTOS/inc中
4. **FreeRTOS移植文件，与编译器相关、keil编译环境**：复制\FreeRTOS\FreeRTOS\Source\portable\RVDS\ARM_CM3中所有文件到FreeRTOS/port中
5. **内存管理相关文件**: 复制\FreeRTOS\Source\portable\MemMang\heap_4.c到FreeRTOS/port中
6. **标准C源文件**:复制\FreeRTOS\Source中的所有.c文件（不包括文夹件）到FreeRTOS/src中
7. **配置文件**： 复制\FreeRTOS\Demo\CORTEX_STM32F103_Keil\FreeRTOSConfig.h 到FreeRTOS/inc中

8. 打开FreeRTOSConfig.h，在最后一个#endif前添加代码

```
#define vPortSVCHandler SVC_Handler
#define xPortSysTickHandler SysTick_Handler
#define xPortPendSVHandler PendSV_Handler
```

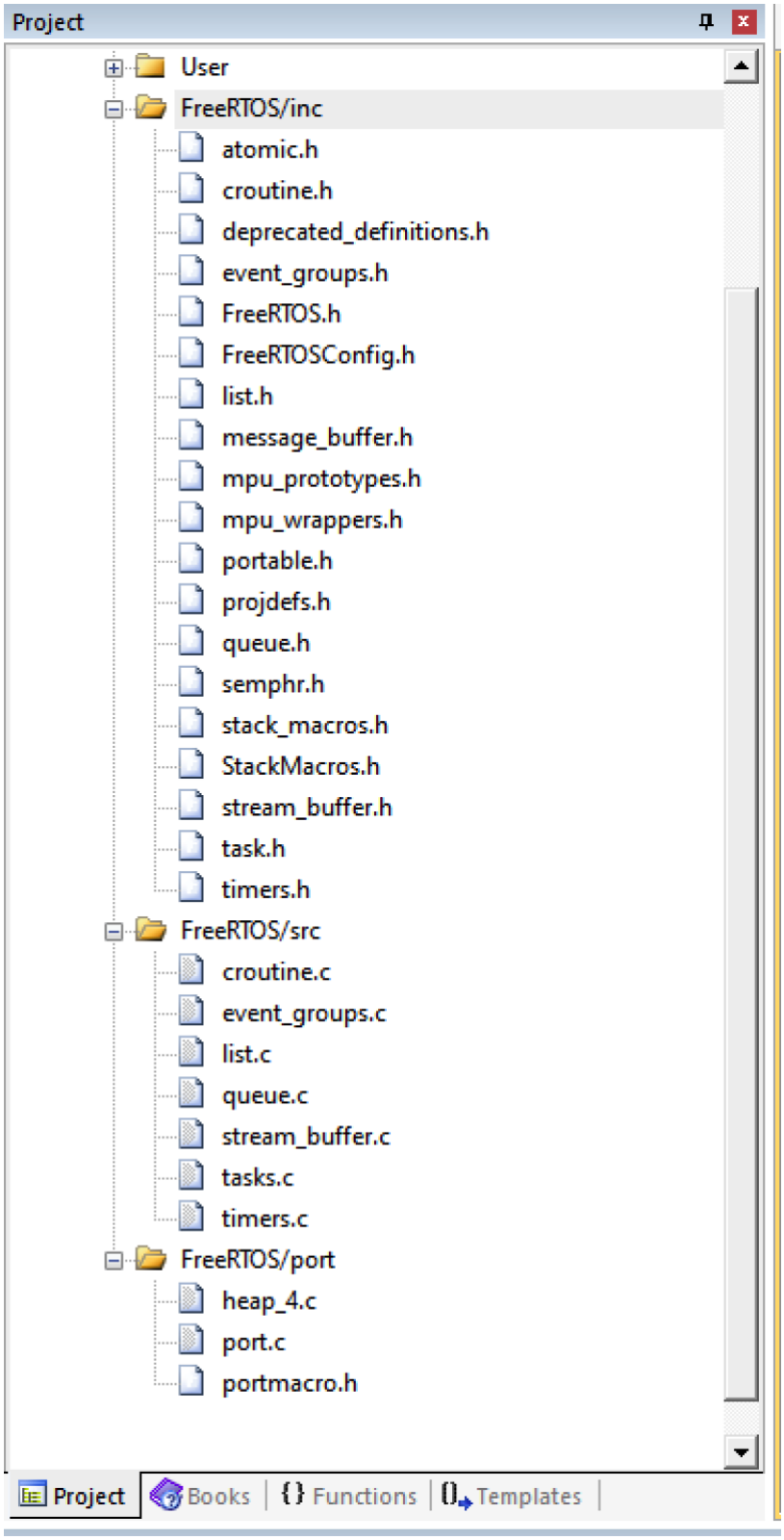
在FreeRTOS中，使用三个宏重写了中断，实现任务调度。

> `vPortSVCHandler` ：用于初始化第一次任务切换。
>
> `xPortSysTickHandler` ：用于定时产生系统滴答，触发任务调度。
>
> `xPortPendSVHandler` ：用于延迟触发上下文切换，确保上下文切换只在需要时执行。

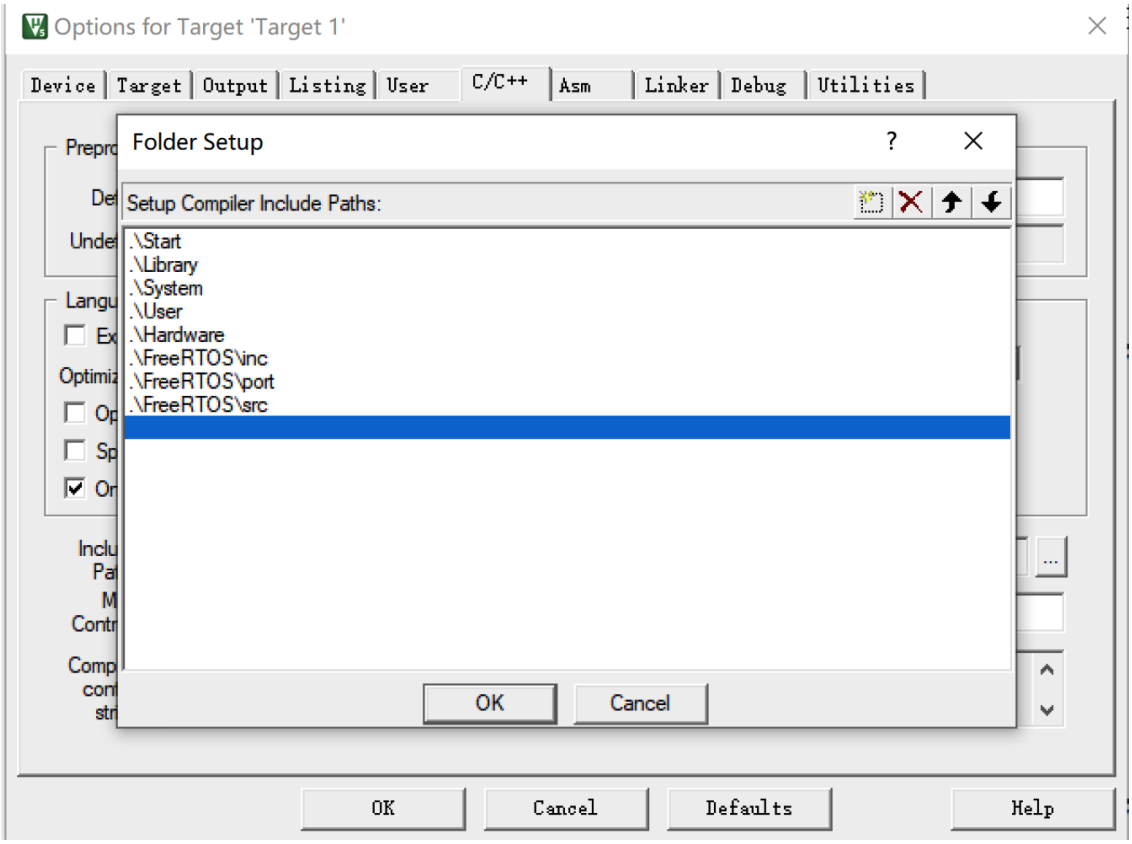
9. 打开stm32f103_it.c文件，注释掉以下三个函数（如果是使用RUN time enviroment工具创建的项目，这个步骤不用做）。

```
//void SVC_Handler(void)
//{
//}
//void PendSV_Handler(void)
//{
//}
//void SysTick_Handler(void)
//{
//}
```

10. 在Keil中添加相应路径下的目录并包含里面的.c和.h的文件



11. 在Keil中添加相应的include Path



12. 编码测试代码

```
#include "stm32f10x.h"
#include "FreeRTOS.h"
#include "task.h"

TaskHandle_t taskHandle;

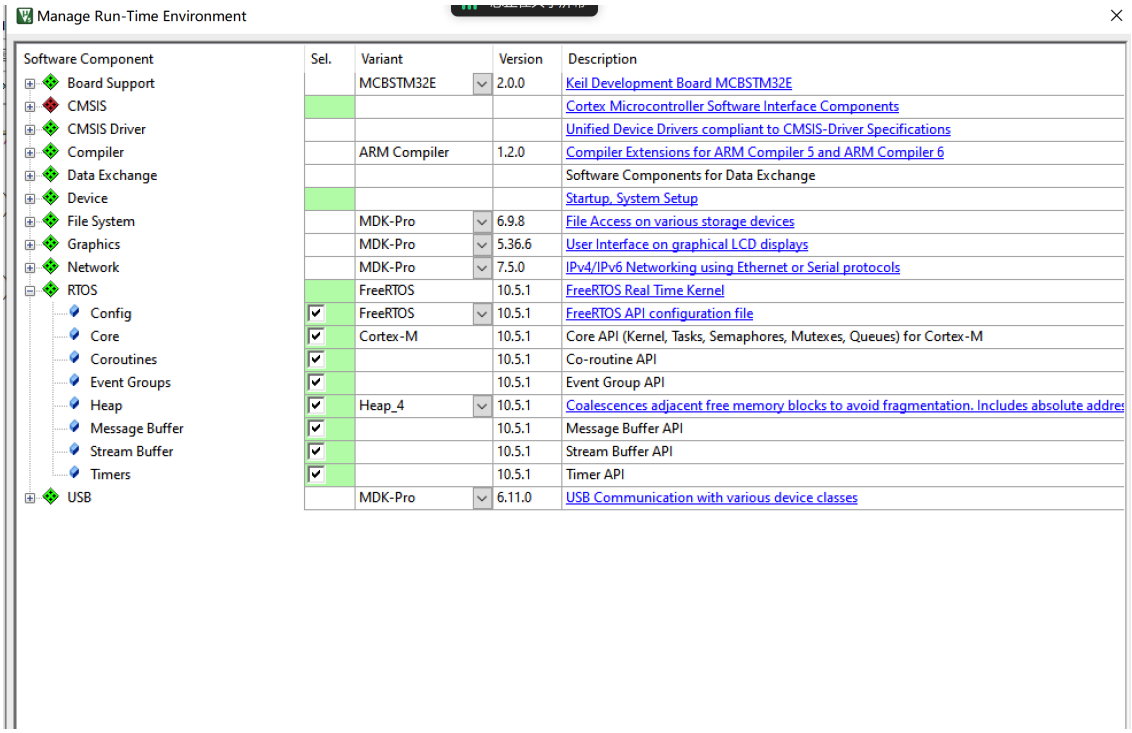
void LED_ON(void *arg)
{
    while(1)
    {
        //向PA0输出高电平-灯灭
        GPIO_WriteBit(GPIOA,GPIO_Pin_0,Bit_RESET);
        vTaskDelay(1000);
        //向PA0输出低电平-灯亮
        GPIO_WriteBit(GPIOA,GPIO_Pin_0,Bit_SET);
        vTaskDelay(1000);
    }
}

int main()
{
    //开启RCC时钟
    RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOA,ENABLE);
    //初始化结构体参数
    GPIO_InitTypeDef GPIO_InitStructA0;
    //指定输出模式为推挽输出
    GPIO_InitStructA0.GPIO_Mode = GPIO_Mode_Out_PP;
    //指定引脚
    GPIO_InitStructA0.GPIO_Pin = GPIO_Pin_0;
```

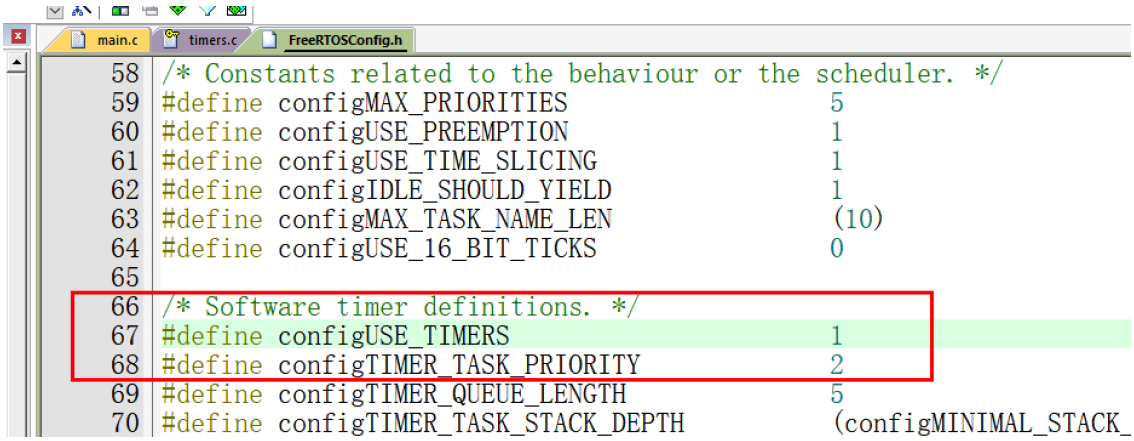
```
    //指定输出频率
    GPIO_InitStructA0.GPIO_Speed = GPIO_Speed_50MHz;
    //初始化GPIO
    GPIO_Init(GPIOA,&GPIO_InitStructA0);
    //创建任务
    xTaskCreate(LED_ON,"led",512,NULL,1,&taskHandle);
    //开始任务调度
    vTaskStartScheduler();
}
```

五、FreeRTOS移植方式二

1. 下载FreeRTOS的支持包并安装
2. 在项目的运行环境中选择FreeRTOS



3. 修改宏定义



```
107   /* 7 priority levels */
108   #define configPRIO_BITS                         4
109 #endif
110
111 /* The lowest interrupt priority that can be used in a call to a "set
112 #define configLIBRARY_LOWEST_INTERRUPT_PRIORITY       0x0F
113
```

六、FreeRTOSConfig.h 配置文件介绍

作用：对FreeRTOS的功能进行配置和裁剪，以及API函数的使能等。

官网中文说明：https://www.freertos.org/zh-cn-cmn-s/Documentation/02-Kernel/03-Supported-devices/02-Customization

```
#ifndef FREERTOS_CONFIG_H
#define FREERTOS_CONFIG_H

/*-----------------------------------------------------------
 * Application specific definitions.
 *
 * These definitions should be adjusted for your particular hardware and
 * application requirements.
 *
 * THESE PARAMETERS ARE DESCRIBED WITHIN THE 'CONFIGURATION' SECTION OF THE
 * FreeRTOS API DOCUMENTATION AVAILABLE ON THE FreeRTOS.org WEB SITE.
 *
 * See http://www.freertos.org/a00110.html
 *----------------------------------------------------------*/

#define configUSE_PREEMPTION        1  // 1: 抢占式调度器, 0: 协程式调度器
#define configUSE_IDLE_HOOK         0  //如果希望使用空闲钩子，请将其设置为 1
(vApplicationIdleHook()函数将在每次空闲任务被执行时调用)
#define configUSE_TICK_HOOK         0  //如果希望使用滴答钩子，请将其设置为 1；
(vApplicationTickHook()函数会在每次 FreeRTOS 的时钟中断（也称为“滴答”中断）触发时被调用)
#define configCPU_CLOCK_HZ          ( ( unsigned long ) 72000000 )  //输入内部时钟的执行频
率（单位为 Hz）
#define configTICK_RATE_HZ          ( ( TickType_t ) 1000 ) //RTOS 滴答中断的频率 ，一个时
间片是1ms
#define configMAX_PRIORITIES        ( 5 ) //最大优先级是5，每个任务均被分配了从 0 到 (
configMAX_PRIORITIES - 1 ) 的优先级，configMAX_PRIORITIES 无法高于 32，优先级数字小表示任务优
先级低
#define configMINIMAL_STACK_SIZE    ( ( unsigned short ) 128 ) //任务使用的堆栈大小，
STM32F103 实际的字节是STACK_SIZE * 4
#define configTOTAL_HEAP_SIZE       ( ( size_t ) ( 15 * 1024 ) ) //堆中可用的 RAM 总量
#define configMAX_TASK_NAME_LEN     ( 16 ) //创建任务时，赋予该任务的描述性名称的最大允许长度
#define configUSE_TRACE_FACILITY    0  //协助执行可视化和跟踪,调试用的
#define configUSE_16_BIT_TICKS      0 //假设滴答频率 为 250Hz,使用 32 位计数器时为 17179869
秒。
#define configIDLE_SHOULD_YIELD     1  //设置为 1， 则在其他具有空闲优先级的任务准备运行时，空
闲任务将立即让出 CPU
```

```
/* Set the following definitions to 1 to include the API function, or zero
to exclude the API function.  启用/关闭 功能 */

#define INCLUDE_vTaskPrioritySet        1
#define INCLUDE_uxTaskPriorityGet       1
#define INCLUDE_vTaskDelete             1
#define INCLUDE_vTaskCleanUpResources   0
#define INCLUDE_vTaskSuspend            1
#define INCLUDE_vTaskDelayUntil         1
#define INCLUDE_vTaskDelay              1   //开启任务延时

/* This is the raw value as per the Cortex-M3 NVIC.  Values can be 255
(lowest) to 0 (1?) (highest). */
#define configKERNEL_INTERRUPT_PRIORITY         255  //设置 RTOS 内核自身使用的中断优先级。一般设置为最低优先级，不至于屏蔽其他优先级程序
/* !!!! configMAX_SYSCALL_INTERRUPT_PRIORITY must not be set to zero !!!!
See http://www.FreeRTOS.org/RTOS-Cortex-M3-M4.html. */
// #define configMAX_SYSCALL_INTERRUPT_PRIORITY     191 /* equivalent to 0xb0, or priority 11. */ // FreeRTOS 的管理的最高优先级
#define configMAX_SYSCALL_INTERRUPT_PRIORITY    5<<4  //建议配置，大于等于5的中断中才能调用FromISR等FreeRTOS API函数


/* This is the value being used as per the ST library which permits 16
priority values, 0 to 15.  This must correspond to the
configKERNEL_INTERRUPT_PRIORITY setting.  Here 15 corresponds to the lowest
NVIC value of 255. */
#define configLIBRARY_KERNEL_INTERRUPT_PRIORITY 15  //FreeRTOS 的库函数会在优先级 15 上运行（最低优先级），并且不会干扰优先级较高的中断（如优先级 0-14 的中断）

//使用FreeRTOS的内置方法处理，把原STM32中的方法注释掉
#define xPortPendSVHandler PendSV_Handler
#define vPortSVCHandler SVC_Handler
#define xPortSysTickHandler SysTick_Handler


#endif /* FREERTOS_CONFIG_H */
```

七、创建任务

0. 延迟函数

- `vTaskDelay():` 相对延时。从执行vTaskDelay()函数开始，直到指定延时的时间结束。
- `xTaskDelayUntil():` 绝对延时。将整个任务的运行周期视为一个整体，适用于需要以固定频率定期执行的任务,比如可以在通讯协议处理上使用。

> 假设有一个定时器，每隔1秒触发一次，希望在每次触发时执行某个任务。如果使用 vTaskDelay 来实现，那么你只能实现任务每秒执行一次，而不能确保任务在每秒的开始时刻执行。
>
> 但如果你使用 xTaskDelayUntil，你可以指定任务在每秒的开始时刻执行，即使任务执行的时间不同。

vTaskDelay用法实例：

```
void LED1Task(void * pvParameters)
{
    while(1){
        LED1_Turn();
        vTaskDelay(500);
    }
}
```

vTaskDelayUtil 用法实例：

```
void LED1Task(void * pvParameters)
{
    TickType_t xLastWakeTime;
    const TickType_t xFrequency = 500;
     xLastWakeTime = xTaskGetTickCount(); // Initialise the xLastWakeTime variable with
the current time.

    while(1){

        LED1_Turn();
        vTaskDelayUntil( &xLastWakeTime, xFrequency );
    }
}
```

1. 任务创建相关函数

API函数	描述
TaskHandle_t	任务引用的类型
xTaskCreate	动态创建一项新任务 并将其添加到准备运行的任务列表中
xTaskCreateStatic	静态创建一项新任务 并将其添加到准备运行的任务列表中
vTaskDelete	删除任务

> 动态创建任务：任务的任务控制块以及任务的栈空间所需的内存，均由 FreeRTOS 从 FreeRTOS 管理的堆中分配。
>
> 静态创建任务：任务的任务控制块以及任务的栈空间所需的内存，需用户分配提供。

2. 动态内存方式

配置文件（FreeRTOSConfig.h）说明：

```
将宏configSUPPORT_DYNAMIC_ALLOCATION 配置为 1。
```

函数说明：

```
BaseType_t xTaskCreate( TaskFunction_t pvTaskCode,  // 指向任务函数的指针

                        const char * const pcName,  //任务名字，最大长度
configMAX_TASK_NAME_LEN

                       const configSTACK_DEPTH_TYPE uxStackDepth, //任务堆栈大小

                       void *pvParameters, //传递给任务函数的参数

                       UBaseType_t uxPriority,//任务优先级，范围：0 ~ configMAX_PRIORITIES
- 1

                      TaskHandle_t *pxCreatedTask //任务句柄指针，就是任务的任务控制块

                     );
```

返回值说明：

```
如果任务创建成功，则返回 pdPASS，
否则返回 errCOULD_NOT_ALLOCATE_REQUIRED_MEMORY。
```

动态内存方式创建任务实例(main.c)

```
#include "stm32f10x.h"                  // Device header
#include "OLED.h"
#include "LED.h"
#include "FreeRTOS.h"
#include "task.h"

//需求：定义2个LED灯，第一个灯亮1秒停一秒，第二个灯亮2秒停2秒
TaskHandle_t LED1TaskHandle;
TaskHandle_t LED2TaskHandle;

void LED1Task(void * pvParameters)
{
    while(1){
        LED1_Turn();
```

```
        vTaskDelay(1000);// 调用任务应阻塞的 tick 周期数 ,vTaskDelayUntil() 是精确计时（如果是协议通讯上用这个）
    }
}

void LED2Task(void * pvParameters)
{
    while(1){
        LED2_Turn();
        vTaskDelay(2000);// 调用任务应阻塞的 tick 周期数
    }
}

int main(void)
{
    //初始化硬件
    OLED_Init();
    OLED_Clear();
    LED_Init();
    BaseType_t taskLED1_CreateType = xTaskCreate(
                                    LED1Task,
                                    "LED1Task",
                                    128,
                                    NULL,
                                    1,
                                    &LED1TaskHandle
                                );
    if(taskLED1_CreateType==pdPASS){
        OLED_ShowString(1,1,"LED1Task pdPASS");
    }

    BaseType_t taskLED2_CreateType = xTaskCreate(
                                    LED2Task,
                                    "LED2Task",
                                    128,
                                    NULL,
                                    1,
                                    &LED2TaskHandle
                                );

    if(taskLED2_CreateType==pdPASS){
        OLED_ShowString(2,1,"LED2Task pdPASS");
    }

    //把任务调度，任务跑起来。
    vTaskStartScheduler();
    //后面的代码执行不到了，可以注释或者删除
//  while(1)
//  {

//
```

```
//  }
}
```

3. 静态内存方式

配置说明

```
#define configSUPPORT_STATIC_ALLOCATION 1
```

函数说明

```
 TaskHandle_t xTaskCreateStatic( TaskFunction_t pxTaskCode,  //任务函数
                                 const char * const pcName,//任务名称
                                 const uint32_t ulStackDepth,//任务堆栈大小
                                 void * const pvParameters,//传递给任务函数的参数
                                 UBaseType_t uxPriority, //任务优先级
                                 StackType_t * const puxStackBuffer,//任务堆栈
                                 StaticTask_t * const pxTaskBuffer ); //任务控制块
```

函数返回值说明

```
如果 puxStackBuffer 和 pxTaskBuffer 均不为 NULL，则创建任务， 并返回任务的句柄。
如果 puxStackBuffer 或 pxTaskBuffer 为 NULL，则不会创建任务， 并返回 NULL。
```

静态内存方式创建任务实例(main.c)

```
#include "stm32f10x.h"                  // Device header
#include "OLED.h"
#include "LED.h"
#include "FreeRTOS.h"
#include "task.h"

//需求：定义2个LED灯，第一个灯亮1秒停一秒，第二个灯亮2秒停2秒
//LED1用静态方式创建

#define STACK_SIZE 200
StaticTask_t xTaskBuffer;  //任务控制块
StackType_t xStack[ STACK_SIZE ]; //任务堆栈
TaskHandle_t LED1TaskHandle;
TaskHandle_t LED2TaskHandle;

// 函数实现：
// 参考https://www.freertos.org/zh-cn-cmn-s/Documentation/02-Kernel/03-Supported-devices/02-Customization#configsupport_static_allocation */
/* configSUPPORT_STATIC_ALLOCATION is set to 1, so the application must provide an
   implementation of vApplicationGetIdleTaskMemory() to provide the memory that is
   used by the Idle task. */
void vApplicationGetIdleTaskMemory( StaticTask_t **ppxIdleTaskTCBBuffer,
                                    StackType_t **ppxIdleTaskStackBuffer,
                                    uint32_t *pulIdleTaskStackSize )
```

```
{
    /* If the buffers to be provided to the Idle task are declared inside this
       function then they must be declared static - otherwise they will be allocated on
       the stack and so not exists after this function exits. */
    static StaticTask_t xIdleTaskTCB;
    static StackType_t uxIdleTaskStack[ configMINIMAL_STACK_SIZE ];

    /* Pass out a pointer to the StaticTask_t structure in which the Idle task's
       state will be stored. */
    *ppxIdleTaskTCBBuffer = &xIdleTaskTCB;

    /* Pass out the array that will be used as the Idle task's stack. */
    *ppxIdleTaskStackBuffer = uxIdleTaskStack;

    /* Pass out the size of the array pointed to by *ppxIdleTaskStackBuffer.
       Note that, as the array is necessarily of type StackType_t,
       configMINIMAL_STACK_SIZE is specified in words, not bytes. */
    *pulIdleTaskStackSize = configMINIMAL_STACK_SIZE;
}



void LED1Task(void * pvParameters)
{
    while(1){
        LED1_Turn();
        vTaskDelay(1000);// 调用任务应阻塞的 tick 周期数 ,vTaskDelayUntil() 是精确计时（如果是协议通讯上用这个）
    }
}

void LED2Task(void * pvParameters)
{
    while(1){
        LED2_Turn();
        vTaskDelay(2000);// 调用任务应阻塞的 tick 周期数
    }
}



int main(void)
{
    //初始化硬件
    OLED_Init();
    OLED_Clear();
    LED_Init();
    LED1TaskHandle = xTaskCreateStatic(
                                    LED1Task,
                                    "LED1Task",
                                    STACK_SIZE,
```

```
                                    NULL,
                                    1,
                                    xStack,
                                    &xTaskBuffer
                                );
    if(LED1TaskHandle!=NULL){
        OLED_ShowString(1,1,"LED1Task pdPASS");
    }

    BaseType_t taskLED2_CreateType = xTaskCreate(
                                    LED2Task,
                                    "LED2Task",
                                    128,
                                    NULL,
                                    1,
                                    &LED2TaskHandle
                                );

    if(taskLED2_CreateType==pdPASS){
        OLED_ShowString(2,1,"LED2Task pdPASS");
    }

    //把任务调度，任务跑起来。
    vTaskStartScheduler();
}
```

如果出现下面的错误提示，则改小FreeRTOS可用的堆空间

```
Build Output
linking...
.\Objects\project.axf: Error: L6406E: No space in execution regions with .ANY selector matching tasks.o(.data).
.\Objects\project.axf: Error: L6406E: No space in execution regions with .ANY selector matching heap_4.o(.data).
.\Objects\project.axf: Error: L6406E: No space in execution regions with .ANY selector matching main.o(.data).
.\Objects\project.axf: Error: L6406E: No space in execution regions with .ANY selector matching port.o(.data).
```

```
#define configTOTAL_HEAP_SIZE       ( ( size_t ) ( 15 * 1024 ) ) //堆中可用的 RAM 总量,改小一些，比如我这里的 15
```

4. 动态创建和静态创建区别

静态创建任务：

1. 需要自己指定堆栈空间
2. 需要自己定义任务控制块

   优点：不易产生内存碎片

   缺点：任务如果删除，无法自动回收内存

动态创建任务：

1. 无需指定堆栈空间，只需要传入一个任务句柄
2. 系统会自动分配任务控制块

   优点：内存分配灵活，节省空间

缺点：容易产生内存碎片

5. 删除任务

配置文件（FreeRTOSConfig.h）说明

```
#define INCLUDE_vTaskDelete             1
```

函数说明

```
void vTaskDelete( TaskHandle_t xTask );//要删除的任务的句柄。如果传递 NULL，会删除任务本身。
```

> 要删除的任务将从所有就绪、 阻塞、挂起和事件列表中移除
>
> 注意：空闲任务负责释放由 RTOS 内核分配给已删除任务的内存。因此，如果应用程序调用了 `vTaskDelete` ，请务必确保空闲任务获得足够的微控制器处理时间。任务代码分配的内存不会自动释放， 应在任务删除之前手动释放。

删除任务实例

key.c

```
#include "stm32f10x.h"                  // Device header
#include "Delay.h"
#include "FreeRTOS.h"
#include "task.h"
//硬件连接：  把按键连接在B0 上的 ，  另一头在GND上
//分析：  上拉输入

void KEY_Init(void)
{
    //1.开启时钟
    RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOB,ENABLE);
    //2.初始化GPIO

    GPIO_InitTypeDef GPIO_InitTypeStructure;
    GPIO_InitTypeStructure.GPIO_Mode = GPIO_Mode_IPU;  //上拉输入
    GPIO_InitTypeStructure.GPIO_Pin = GPIO_Pin_0;
    GPIO_InitTypeStructure.GPIO_Speed = GPIO_Speed_50MHz;
    GPIO_Init(GPIOB,&GPIO_InitTypeStructure);


}

//返回1就是按了按键，返回0 就是没有按按键
uint8_t Key_Press(void)
{
    if(GPIO_ReadInputDataBit(GPIOB,GPIO_Pin_0) == 0)
    {
        vTaskDelay(10);//滤除按下时的抖动 ----------- 注意这里
        while(GPIO_ReadInputDataBit(GPIOB,GPIO_Pin_0) == 0) ; //可能按住不松手
        vTaskDelay(10);//滤除松手时的抖动 --- --------注意这里
```

```
        return 1;
    }

    return 0;
}
```

main.c

```
#include "stm32f10x.h"                  // Device header
#include "OLED.h"
#include "LED.h"
#include "KEY.h"
#include "FreeRTOS.h"
#include "task.h"

//需求：定义2个LED灯，第一个灯亮1秒停一秒，第二个灯亮2秒停2秒
//LED1用静态方式创建
//按下按键，删除任务LED1TaskHandle

#define STACK_SIZE 200
StaticTask_t xTaskBuffer;  //任务控制块
StackType_t xStack[ STACK_SIZE ]; //任务堆栈
TaskHandle_t LED1TaskHandle;
TaskHandle_t LED2TaskHandle;
TaskHandle_t KeyTaskHandle;

// 函数实现：
void vApplicationGetIdleTaskMemory( StaticTask_t **ppxIdleTaskTCBBuffer,
                                    StackType_t **ppxIdleTaskStackBuffer,
                                    uint32_t *pulIdleTaskStackSize )
{
    static StaticTask_t xIdleTaskTCB;
    static StackType_t uxIdleTaskStack[ configMINIMAL_STACK_SIZE ];
    *ppxIdleTaskTCBBuffer = &xIdleTaskTCB;
    *ppxIdleTaskStackBuffer = uxIdleTaskStack;
    *pulIdleTaskStackSize = configMINIMAL_STACK_SIZE;
}



void LED1Task(void * pvParameters)
{
    while(1){
        LED1_Turn();
        vTaskDelay(1000);// 调用任务应阻塞的 tick 周期数 ,vTaskDelayUntil() 是精确计时（如果是协议通讯上用这个）
    }
}
```

```
void LED2Task(void * pvParameters)
{
    while(1){
        LED2_Turn();
        vTaskDelay(2000);// 调用任务应阻塞的 tick 周期数
    }
}

void KeyTask(void * pvParameters)
{
    while(1)
    {
        if(Key_Press()==1)
        {
            vTaskDelete(LED1TaskHandle);
        }
    }
}

int main(void)
{
    //初始化硬件
    OLED_Init();
    OLED_Clear();
    LED_Init();
    KEY_Init();

    LED1TaskHandle = xTaskCreateStatic(
                                    LED1Task,
                                    "LED1Task",
                                    STACK_SIZE,
                                    NULL,
                                    1,
                                    xStack,
                                    &xTaskBuffer
                                );
    if(LED1TaskHandle!=NULL){
        OLED_ShowString(1,1,"LED1Task pdPASS");
    }

    BaseType_t taskLED2_CreateType = xTaskCreate(
                                    LED2Task,
                                    "LED2Task",
                                    128,
                                    NULL,
                                    1,
                                    &LED2TaskHandle
                                );

    if(taskLED2_CreateType==pdPASS){
```

```
        OLED_ShowString(2,1,"LED2Task pdPASS");
    }

    BaseType_t taskKey_CreateType = xTaskCreate(
                                    KeyTask,
                                    "KeyTask",
                                    128,
                                    NULL,
                                    1,
                                    &KeyTaskHandle
                                );

    if(taskKey_CreateType==pdPASS){
        OLED_ShowString(3,1,"KeyTask pdPASS");
    }

    //把任务调度，任务跑起来。
    vTaskStartScheduler();
}
```

八、任务挂起与恢复

1.任务挂起 vTaskSuspend

配置文件（FreeRTOSConfig.h）说明

```
# #define INCLUDE_vTaskSuspend   1 //必须定义为 1，才可使用此函数
```

函数

```
void vTaskSuspend( TaskHandle_t xTaskToSuspend ); //无论任务优先级如何，任务被挂起后将永远无法获取任何微控制器处理时间。
```

2.任务恢复vTaskResume

配置文件（FreeRTOSConfig.h）说明

```
#define INCLUDE_vTaskSuspend   1 //必须定义为 1，才可使用此函数
```

函数

```
void vTaskResume( TaskHandle_t xTaskToResume );//不论任务被使用 vTaskSuspend() 挂起多少次，只需调用 vTaskResume() 一次，即可使其继续执行
```

3.任务从中断恢复 xTaskResumeFromISR

配置文件（FreeRTOSConfig.h）说明

```
#define INCLUDE_vTaskSuspend    1
#define INCLUDE_xTaskResumeFromISR  1
```

函数

```
BaseType_t xTaskResumeFromISR( TaskHandle_t xTaskToResume ); //要恢复的任务句柄。
```

> 返回值：
>
> - 如果恢复任务导致上下文切换，则返回 *pdTRUE* ，
> - 否则返回 *pdFALSE* 。
> - ISR 使用此信息来确定 ISR 之后是否需要上下文切换。
> - 切换方法：portYIELD_FROM_ISR(pdTRUE)

4.实验

实验现象：按下key1 挂起LED1任务，按下key2 恢复LED1任务

key.c

```
#include "stm32f10x.h"                  // Device header
#include "Delay.h"
#include "FreeRTOS.h"
#include "task.h"
//硬件连接：  把按键连接在B0 上的 ，  另一头在GND上
//分析：  上拉输入

void KEY_Init(void)
{
    //1.开启时钟
    RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOB,ENABLE);
    //2.初始化GPIO

    GPIO_InitTypeDef GPIO_InitTypeStructure;
    GPIO_InitTypeStructure.GPIO_Mode = GPIO_Mode_IPU;  //上拉输入
    GPIO_InitTypeStructure.GPIO_Pin = GPIO_Pin_0|GPIO_Pin_10;
    GPIO_InitTypeStructure.GPIO_Speed = GPIO_Speed_50MHz;
    GPIO_Init(GPIOB,&GPIO_InitTypeStructure);


}

//返回1就是按了按键，返回0 就是没有按按键
uint8_t Key1_Press(void)
{
    if(GPIO_ReadInputDataBit(GPIOB,GPIO_Pin_0) == 0)
```

```
    {
        vTaskDelay(10);//滤除按下时的抖动
        while(GPIO_ReadInputDataBit(GPIOB,GPIO_Pin_0) == 0) ; //可能按住不松手
        vTaskDelay(10);//滤除松手时的抖动
        return 1;
    }

    return 0;
}

//返回1就是按了按键，返回0 就是没有按按键
uint8_t Key2_Press(void)
{
    if(GPIO_ReadInputDataBit(GPIOB,GPIO_Pin_10) == 0)
    {
        vTaskDelay(10);//滤除按下时的抖动
        while(GPIO_ReadInputDataBit(GPIOB,GPIO_Pin_10) == 0) ; //可能按住不松手
        vTaskDelay(10);//滤除松手时的抖动
        return 1;
    }
    return 0;
}
```

main.c

```
#include "stm32f10x.h"                  // Device header
#include "OLED.h"
#include "LED.h"
#include "KEY.h"
#include "FreeRTOS.h"
#include "task.h"

//需求：按下key1 挂起LED1，按下key2 恢复LED1

TaskHandle_t LED1TaskHandle;
TaskHandle_t Key1TaskHandle;
TaskHandle_t Key2TaskHandle;

void LED1Task(void * pvParameters)
{
    while(1){
        LED1_Turn();
        vTaskDelay(500);
    }
}

void Key1Task(void * pvParameters)
{
    while(1)
    {
        if(Key1_Press()==1)
```

```
        {
            vTaskSuspend(LED1TaskHandle);
        }
    }
}


void Key2Task(void * pvParameters)
{
    while(1)
    {
        if(Key2_Press()==1)
        {
            vTaskResume(LED1TaskHandle);
        }
    }
}
int main(void)
{
    //初始化硬件
    OLED_Init();
    OLED_Clear();
    LED_Init();
    KEY_Init();

    xTaskCreate(LED1Task,
                "LED1Task",
                128,
                NULL,
                1,
                &LED1TaskHandle
                );
    xTaskCreate(Key1Task,
                "Key1Task",
                128,
                NULL,
                1,
                &Key1TaskHandle
                );
    xTaskCreate(Key2Task,
                "Key2Task",
                128,
                NULL,
                1,
                &Key2TaskHandle
                );

    //把任务调度，任务跑起来。
    vTaskStartScheduler();
}
```

九、中断、临界区

1.FreeRTOS的中断管理

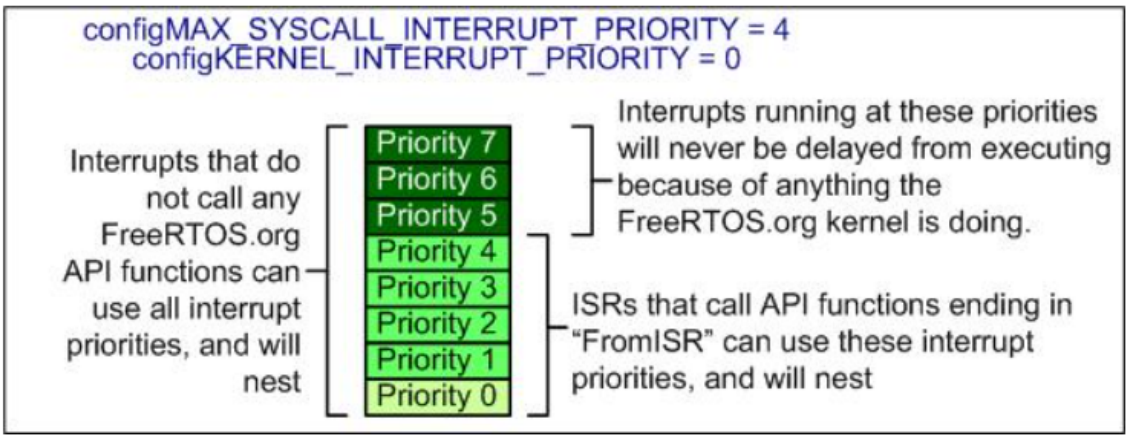
在STM32中，中断优先级是通过中断优先级配置寄存器的 高4位 [7:4] 来配置的。因此STM32支持最多16级中断优先级，其中数值越小表示优先级越高，即更紧急的中断。（FreeRTOS任务调度的任务优先级相反，是数值越大越优先）

FreeRTOS可以与STM32原生的中断机制结合使用，但它提供了自己的中断管理机制，主要是为了提供更强大和灵活的任务调度和管理功能。

FreeRTOS中，将PendSV和SysTick设置最低中断优先级（数值最大，15），保证系统任务切换不会阻塞系统其他中断的响应。

FreeRTOS利用**BASEPRI**寄存器实现中断管理，屏蔽优先级低于某一个阈值的中断。比如： **BASEPRI**设置为0x50（只看高四位，也就是5），代表中断优先级在5~15内的均被屏蔽，0~4的中断优先级正常执行。

例如，假设一个微控制器具有 8 个中断优先级：0 为最低优先级， 7 为最高优先级。如果 将两个配置常量设置为 4 和 0，下图描述了在各个优先级上 可以执行以及不能执行的操作：



在中断服务函数中调用FreeRTOS的API函数需注意：

> 中断服务函数的优先级需在FreeRTOS所管理的范围内，阈值由configMAX_SYSCALL_INTERRUPT_PRIORITY指定。
>
> 建议将**所有优先级位指定为抢占优先级位**，方便FreeRTOS管理。（配置NVIC_PRIORITYGROUP_4优先级组)
>
> 在中断服务函数里边需调用FreeRTOS的API函数，必须使用带“**FromISR**”后缀的函数。

2.FreeRTOS的开关中断

FreeRTOS 开关中断函数其实是宏定义，在 portmacro.h 中有定义，如下：

```
#define portDISABLE_INTERRUPTS()          vPortRaiseBASEPRI()
#define portENABLE_INTERRUPTS()           vPortSetBASEPRI( 0 )
```

> 调用portENABLE_INTERRUPTS() 它, FreeRTOS会 打开 管理的所有中断
>
> 调用portDISABLE_INTERRUPTS() 它, FreeRTOS会 关闭 管理的所有中断

3. 使用临界区 CRITICAL

临界段用一句话概括就是一段在执行的时候不能被 中断 的代码段。 在 FreeRTOS 里面，这个临界段最常出现的就是对全局变量的操作。

那么什么情况下临界段会被打断？一个是 系统调度 ，还有一个就是 外部中断 。在FreeRTOS，系统调度，最终也是产生 PendSV 中断，在 PendSV Handler 里面实现任务的切换，所以还是可以归结为中断。 既然这样， FreeRTOS 对临界段的保护最终还是回到对中断的开和关的控制。

FreeRTOS 关中断的函数在 portmacro.h 中定义， 分不带返回值和带返回值两种：

- 不带返回值的关中断函数，不能嵌套，不能在中断里面使用。 portDISABLE_INTERRUPTS()
- 带返回值的关中断函数，可以嵌套，可以在中断里面使用。
  portSET_INTERRUPT_MASK_FROM_ISR()

FreeRTOS 开中断的函数在 portmacro.h 中定义

- 不带中断保护的开中断函数 portENABLE_INTERRUPTS()
- 带中断保护的开中断函数 portCLEAR_INTERRUPT_MASK_FROM_ISR(x)

进入和退出临界段宏定义

```
#define taskENTER_CRITICAL() portENTER_CRITICAL()   //进入临界段。
#define taskENTER_CRITICAL_FROM_ISR() portSET_INTERRUPT_MASK_FROM_ISR()  //进入临界段（中断级）。

#define taskEXIT_CRITICAL() portEXIT_CRITICAL()  //退出临界段。
#define taskEXIT_CRITICAL_FROM_ISR( x ) portCLEAR_INTERRUPT_MASK_FROM_ISR( x ) //退出临界段（中断级）。
```

> 进入和退出临界段的宏分中断保护版本和非中断版本， 但最终都是通过开/关中断来实现。
>
> 进入和退出临界段是成对使用的。
>
> 每只有当临界段代码都退出时，中断才会被重新使能。

示例代码：SerialTask1任务通过串口输出信息，SerialTask2Task任务也通过串口同时输出信息

```
#include "stm32f10x.h"                  // Device header
#include "Serial2.h"
#include "FreeRTOS.h"
#include "task.h"

//需求：SerialTask1任务通过串口输出信息，SerialTask2Task任务也通过串口同时输出信息

TaskHandle_t SerialTask1Handle;
TaskHandle_t SerialTask2Handle;

TaskHandle_t CreateTaskHandle;//创建任务的句柄


void SerialTask1Task(void * pvParameters)
{
```

```
    while(1){

        Serial2_SendString((char *)pvParameters);

        vTaskDelay(1000);
    }
}


void SerialTask2Task(void * pvParameters)
{
    while(1){

        Serial2_SendString((char *)pvParameters);

        vTaskDelay(1000);
    }
}


void CreateTask(void * pvParameters)
{

    xTaskCreate(SerialTask1Task,"SerialTask1Task",128,"hello\r\n",1,&SerialTask1Handle);
    xTaskCreate(SerialTask2Task,"SerialTask2Task",128,"woniu\r\n",1,&SerialTask2Handle);

    vTaskDelete( NULL );//删除自己，别忘记
}

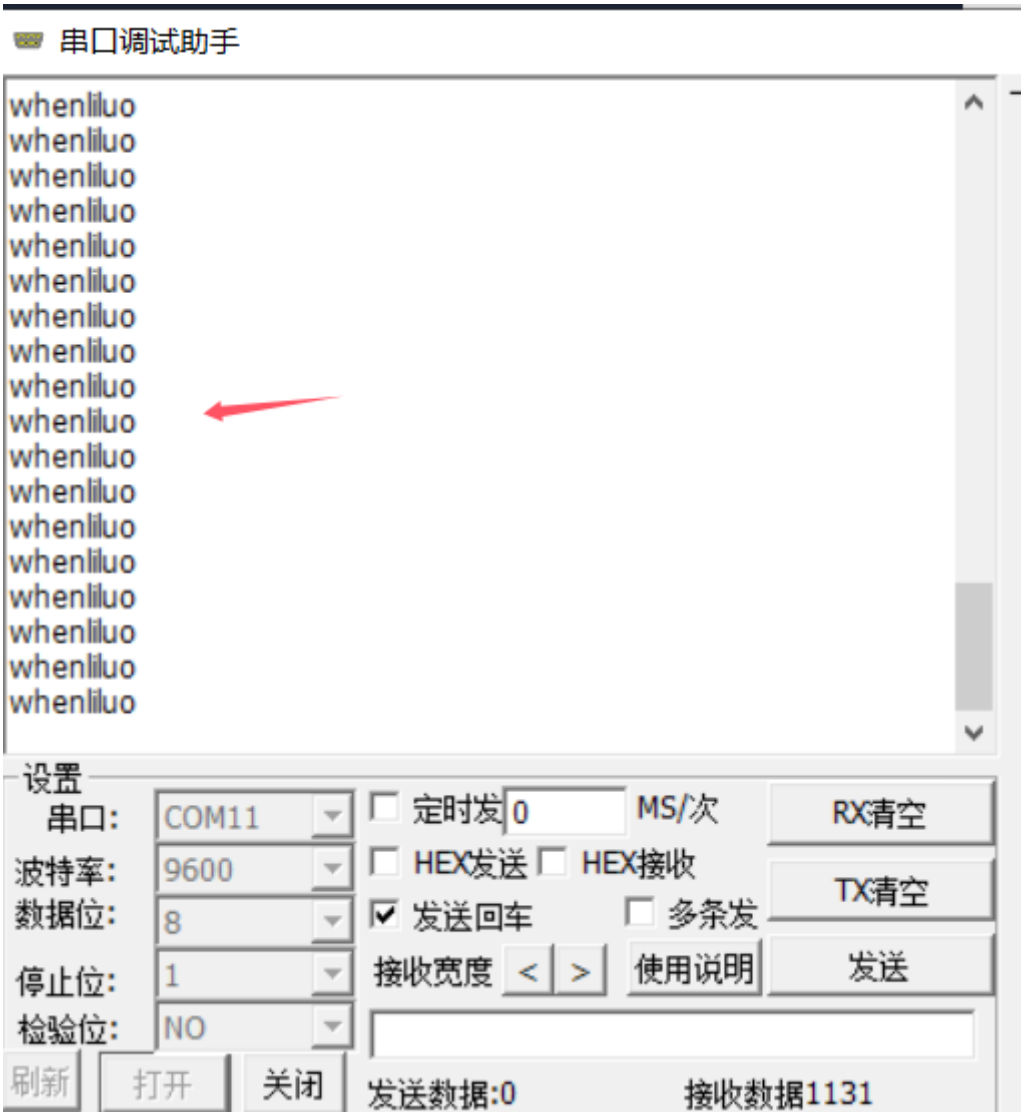


int main(void)
{
    //初始化硬件
    Serial2_Init();

    //创建任务
    xTaskCreate(CreateTask,"CreateTask",128,NULL,3,&CreateTaskHandle);

    //把任务跑起来。
    vTaskStartScheduler();
}
```

由于2个任务同时操作串口，出现的问题如下图,输出的字符是 错乱 的。



接下来通过加入临界区，解决上面的问题，当SerialTask1任务使用串口时，SerialTask2任务需要等待SerialTask1退出临界区

mian.c

```
#include "stm32f10x.h"                  // Device header
#include "Serial2.h"
#include "FreeRTOS.h"
#include "task.h"

//需求：SerialTask1任务通过串口输出信息，SerialTask2Task任务也通过串口同时输出信息

TaskHandle_t SerialTask1Handle;
TaskHandle_t SerialTask2Handle;

TaskHandle_t CreateTaskHandle;//创建任务的句柄


void SerialTask1Task(void * pvParameters)
{
    while(1){
        taskENTER_CRITICAL();//进入临界区（别的任务和中断没法打断临界区的代码执行）
        Serial2_SendString((char *)pvParameters);
        taskEXIT_CRITICAL();//退出临界区
```

```
        vTaskDelay(1000);
    }
}


void SerialTask2Task(void * pvParameters)
{
    while(1){
        taskENTER_CRITICAL();//进入临界区
        Serial2_SendString((char *)pvParameters);
        taskEXIT_CRITICAL();//退出临界区
        vTaskDelay(1000);
    }
}


void CreateTask(void * pvParameters)
{
    taskENTER_CRITICAL(); //创建任务也可以加上临界区
    xTaskCreate(SerialTask1Task,"SerialTask1Task",128,"hello\r\n",1,&SerialTask1Handle);
    xTaskCreate(SerialTask2Task,"SerialTask2Task",128,"woniu\r\n",1,&SerialTask2Handle);
    vTaskDelete( NULL );//删除自己，别忘记
    taskEXIT_CRITICAL();
}

int main(void)
{
    //初始化硬件
    Serial2_Init();

    //创建任务
    xTaskCreate(CreateTask,"CreateTask",128,NULL,3,&CreateTaskHandle);

    //把任务跑起来。
    vTaskStartScheduler();
}
```



十、消息队列

> 注意：
>
> 1. 使用消息队列传输数据时，是在复制数据。
>
>   解决方案：传递数据时，不要直接传递数据，而是传递指针
> 2. 如果要传输不同类型的数据怎么传?
>
>   解决方案：使用结构体

队列又称消息队列，是一种常用于任务间通信的数据结构， 队列可以在任务与任务间、 中断和任务间传递信息，实现了任务接收来自其他任务或中断的**不固定长度**的消息。

FreeRTOS 中使用队列数据结构实现任务异步通信工作，具有如下特性：

- 消息支持先进先出方式（FIFO）排队，支持异步读写工作方式。
- 读写队列均支持超时机制。
- 消息支持后进先出方式排队， 往队首发送消息（LIFO） 。
- 可以允许不同长度（不超过队列节点最大值）的任意类型消息。
- 一个任务能够从任意一个消息队列接收和发送消息。

- 多个任务能够从同一个消息队列接收和发送消息。
- 当队列使用结束后，可以通过删除队列函数进行删除。

消息队内存大小=消息队列控制块大小加上+（单个消息空间大小*消息队列长度的乘积）

任务或者中断服务程序都可以给消息队列发送消息， 当发送消息时， 如果队列未满或者允许覆盖入队 FreeRTOS 会将消息拷贝到消息队列队尾，否则，会根据用户指定的阻塞超时时间进行阻塞，在这段时间中，如果队列一直不允许入队，该任务将保持阻塞状态以等待队列允许入队。当其它任务从其等待的队列中读取入了数据（队列未满），该任务将自动由阻塞态转移为就绪态。当等待的时间超过了指定的阻塞时间，即使队列中还不允许入队，任务也会自动从阻塞态转移为就绪态，此时发送消息的任务或者中断程序会收到一个错误码 **errQUEUE_FULL**。

1、消息队列的阻塞机制

而在发送消息操作的时候，为了保护数据，**当且仅当队列允许入队的时候**，发送者才能成功发送消息；队列中无可用消息空间时，说明消息队列已满，此时，系统会根据用户指定的阻塞超时时间将任务阻塞，在指定的超时时间内如果还不能完成入队操作，发送消息的任务或者中断服务程序会收到一个错误码 errQUEUE_FULL，然后解除阻塞状态；当然， 只有在任务中发送消息才允许进行阻塞状态，而在中断中发送消息不允许带有阻塞机制的，需要调用在中断中发送消息的 API 函数接口，因为发送消息的上下文环境是在中断中，不允许有阻塞的情况。

假如有多个任务阻塞在一个消息队列中，那么这些阻塞的任务将按照任务优先级进行排序，优先级高的任务将优先获得队列的访问权。

2、常用api

> //消息队列创建函数
> xQueueCreate(UBaseType_t uxQueueLength, UBaseType_t uxItemSize);
>
> //消息队列静态创建函数
> xQueueCreateStatic(UBaseType_t uxQueueLength, UBaseType_t uxItemSize, uint8_t *pucQueueStorageBuffer, StaticQueue_t *pxQueueBuffer);
>
> 消息队列删除函数 vQueueDelete(),根据消息队列句柄直接删除的
>
> 向消息队列发送消息函数 ,如果队列未满或者允许覆盖入队，FreeRTOS 会将消息拷贝到消息队列队尾，否则，会根据用户指定的阻塞超时时间进行阻塞
> 消息队列发送函数有好几个， 都是使用宏定义进行展开的， 有些只能在任务调用，有些只能在中断中调用:
> xQueueSend()与 xQueueSendToBack() 相等
> xQueueSendFromISR()与 xQueueSendToBackFromISR() 相等，带中断保护
> xQueueSendToFront()
> xQueueSendToFrontFromISR()
>
> 从消息队列读取消息函数
> xQueueReceive() 从一个队列中接收消息并把消息从队列中删除 。 接收的消息是以拷贝的形式进行的， 所以我们必须提供一个足够大空间的缓冲区。
> xQueuePeek() 函数接收消息完毕不会删除消息队列中的消息
> xQueueReceiveFromISR()与 xQueuePeekFromISR()

4、注意事项

1. 使用 xQueueSend()、 xQueueSendFromISR()、 xQueueReceive()等这些函数之前应先创建需消息队列，并根据队列句柄进行操作。
2. 队列读取采用的是先进先出（FIFO）模式，会先读取先存储在队列中的数据。当然也 FreeRTOS 也支持后进先出（LIFO）模式，那么读取的时候就会读取到后进队列的数据。
3. 在获取队列中的消息时候，我们必须要定义一个存储读取数据的地方，并且该数据区域大小不小于消息大小，否则，很可能引发地址非法的错误。
4. 无论是发送或者是接收消息都是以拷贝的方式进行， 如果消息过于庞大，可以将消息的地址作为消息进行发送、接收。
5. 队列是具有自己独立权限的内核对象，并不属于任何任务。所有任务都可以向同一队列写入和读出。一个队列由多任务或中断写入是经常的事，但由多个任务读出倒是用的比较少。

示例代码

```
#include "stm32f10x.h"                  // Device header
#include "Delay.h"
#include "OLED.h"
#include "LED.h"
#include "Serial2.h"
#include "FreeRTOS.h"
#include "task.h"
#include "queue.h"

//需求：任务1和任务2都需要发送串口数据，使用串口队列发送

TaskHandle_t SerialTask1Handle; //需要发送字符串
TaskHandle_t SerialTask2Handle;//需要发送字符串
TaskHandle_t SerialPrintHandle;//利用串口资源发送字符串


TaskHandle_t CreateTaskHandle;//创建任务的句柄

//创建队列句柄
QueueHandle_t xQueueSerial;


void SerialTask1Task(void * pvParameters)
{
    while(1){

        xQueueSend(xQueueSerial,(char *)pvParameters,portMAX_DELAY);//portMAX_DELAY 一直等
        vTaskDelay(1000);
    }
}


void SerialTask2Task(void * pvParameters)
{
    while(1){

        xQueueSend(xQueueSerial,(char *)pvParameters,portMAX_DELAY);//portMAX_DELAY 一直等
```

```
        vTaskDelay(1000);
    }
}


//串口消息发送任务
void SerialPrintTask(void * pvParameters)
{
    char mesg[200];
    while(1)
    {
        //队列中取出要发送的字符串

        xQueueReceive(xQueueSerial,mesg,portMAX_DELAY);

        taskENTER_CRITICAL();
        Serial2_SendString(mesg);
        taskEXIT_CRITICAL();

    }
}


void CreateTask(void * pvParameters)
{

    xTaskCreate(SerialTask1Task,"SerialTask1Task",128,"hello\r\n",1,&SerialTask1Handle);
    xTaskCreate(SerialTask2Task,"SerialTask2Task",128,"woniu\r\n",1,&SerialTask2Handle);
    xTaskCreate(SerialPrintTask,"SerialPrintTask",128,NULL,1,&SerialTask2Handle);

    vTaskDelete( NULL );//删除自己，别忘记
}



int main(void)
{
    //初始化硬件
    OLED_Init();
    OLED_Clear();
    LED_Init();

    Serial2_Init();

    //创建队列
    char queuebuff[200];
    xQueueSerial = xQueueCreate( 20, sizeof( queuebuff ) );//放入要发送串口的消息，队列长度
20，每个元素大小

    //创建任务
    xTaskCreate(CreateTask,"CreateTask",128,NULL,3,&CreateTaskHandle);
```

```
    //把任务跑起来。
    vTaskStartScheduler();

}
```

十一、信号量

信号量可以实现任务之间同步或临界资源的互斥访问， 常用于协助一组相互竞争的任务来访问临界资源。

信号量是一个**非负整数**，所有获取它的任务都会将该整数减一（获取它当然是为了使用资源），当该整数值为零时，所有试图获取它的任务都将处于阻塞状态。通常一个信号量的计数值用于对应有效的资源数，表示剩下的可被占用的互斥资源数。其值的含义分两种情况：

- 0： 表示没有积累下来的释放信号量操作，且有可能有在此信号量上阻塞的任务。
- 正值，表示有一个或多个释放信号量操作。

信号量	作用
计数信号量	判断是信号量是否为空，就是获取和释放是不是相等
二值信号量	二值信号量其实就是一个只有一个队列项的队列
互斥信号量	互斥信号量有优先级继承的机制，所以只能用在任务中，不能用于中断服务函数
递归互斥信号量	可以同时获取同一信号量

1、二值信号量

二值信号量既可以用于临界资源访问也可以用于同步功能。二值信号量的最大可用信号量个数为 1。

用作同步时：信号量在创建后应被置为**空**，任务 1 获取信号量而进入阻塞，任务 2 在某种条件发生后，释放信号量，于是任务 1 获得信号量得以**进入就绪态**，如果任务 1 的优先级是最高的，那么就会立即切换任务，从而达到了两个任务间的同步。同样的，在中断服务函数中释放信号量， 任务 1 也会得到信号量，从而达到任务与中断间的**同步**。

可以将二值信号量看作只有一个消息的队列， 因此这个队列只能为空或满（因此称为二值），我们在运用的时候只需要知道队列中是否有消息即可，而无需关注消息是什么。

二值信号量获取， 任何任务都可以从创建的二值信号量资源中获取一个二值信号量，获取成功则返回正确， 否则任务会根据用户指定的阻塞超时时间来等待其它任务/中断释放信号量。 在等待这段时间，系统将任务变成阻塞态， 任务将被挂到该信号量的阻塞等待列表中。

2、计数信号量

计数信号量则可以被认为长度大于1的队列，信号量使用者依然不必关心存储在队列中的消息，只需关心队列是否有消息即可。

在实际的使用中，我们常将计数信号量用于事件计数与资源管理，信号量的计数值则表示还有多少个事件没被处理

每当某个事件发生时，任务或者中断将释放一个信号量（信号量计数值**加 1**）

当处理被事件时（一般在任务中处理），处理任务会取走该信号量（信号量计数值**减 1**）

任务必须先获取到信号量才能获取资源访问权，当信号量的计数值为零时表示系统没有可用的资源，但是要注意，在使用完资源的时候必须归还信号量，否则当计数值为 0的时候任务就无法访问该资源了。访问的任务数达到可支持的最大数目时，会阻塞其他试图获取该信号量的任务，直到有任务释放了信号量。

3、互斥信号量

互斥信号量其实是特殊的二值信号量，由于其特有的优先级继承机制从而使它更适用于简单互锁，也就是保护临界资源。

用作互斥时，信号量创建后可用信号量个数应该是满的， 任务在需要使用临界资源时，（临界资源是指任何时刻只能被一个任务访问的资源），先获取互斥信号量，使其变空，这样其他任务需要使用临界资源时就会因为无法获取信号量而进入阻塞，从而保证了临界资源的安全。

在操作系统中，我们使用信号量的很多时候是为了给临界资源建立一个标志，信号量表示了该临界资源被占用情况。这样，当一个任务在访问临界资源的时候，就会先对这个资源信息进行查询，从而在了解资源被占用的情况之后，再做处理，从而使得临界资源得到有效的保护。

4、递归信号量

可以重复获取调用的信号量，本来按照信号量的特性，每获取一次可用信号量个数就会减少一个，但是递归则不然， 对于已经获取递归互斥量的任务**可以重复获取该递归互斥量**， 该任务拥有递归信号量的所有权。 任务成功获取几次递归互斥量， **就要返还几次**，**在此之前递归互斥量都处于无效状态**， 其他任务无法获取， 只有持有递归信号量的任务才能获取与释放。

5、常用api

信号量	创建	获取	释放
二值信号量	xSemaphoreCreateBinary()	xSemaphoreTake()	xSemaphoreGive()
计数信号量	xSemaphoreCreateCounting()	xSemaphoreTake()	xSemaphoreGive()
互斥信号量	xSemaphoreCreateMutex()	xSemaphoreTake()	xSemaphoreGive()
递归互斥信号量	xSemaphoreCreateRecursiveMutex()	xSemaphoreTakeRecursive()	xSemaphoreGiveRecursive()

ucQueueType 可选类型

```
#define queueQUEUE_TYPE_BASE ( ( uint8_t ) 0U )
#define queueQUEUE_TYPE_SET ( ( uint8_t ) 0U )
#define queueQUEUE_TYPE_MUTEX ( ( uint8_t ) 1U )
#define queueQUEUE_TYPE_COUNTING_SEMAPHORE ( ( uint8_t ) 2U )
#define queueQUEUE_TYPE_BINARY_SEMAPHORE ( ( uint8_t ) 3U )
#define queueQUEUE_TYPE_RECURSIVE_MUTEX ( ( uint8_t ) 4U )
```

其它

信号量删除函数 vSemaphoreDelete()

vSemaphoreDelete(

SemaphoreHandle_t xSemaphore

)用于删除一个信号量，包括二值信号量，计数信号量，互斥量和递归互斥量

信号量释放函数

当信号量有效的时候， 任务才能获取信号量

xSemaphoreGive()（任务）

xSemaphoreGiveFromISR()（中断）

信号量获取函数

xSemaphoreTake()（任务）

xSemaphoreTakeFromISR()（中断）

十二、互斥量

互斥量又称互斥信号量（本质是信号量），是一种特殊的二值信号量，它和信号量不同的是，它支持互斥量所有权、递归访问以及防止优先级翻转的特性，用于实现对临界资源的独占式处理。

任意时刻互斥量的状态只有两种，**开锁或闭锁**。

当互斥量被任务持有时，该互斥量处于闭锁状态，这个任务获得互斥量的所有权。

当该任务释放这个互斥量时，该互斥量处于开锁状态， 任务失去该互斥量的所有权。

当一个任务持有互斥量时，其他任务将不能再对该互斥量进行开锁或持有。 持有该互斥量的任务也能够再次获得这个锁而不被挂起，这就是递归访问，也就是递归互斥量的特性，这个特性与一般的信号量有很大的不同，在信号量中，由于已经不存在可用的信号量， 任务递归获取信号量时会发生主动挂起任务最终形成死锁。

用于互锁的互斥量可以充当保护资源的令牌， 当一个任务希望访问某个资源时，它必须先获取令牌。当任务使用完资源后，必须还回令牌，以便其它任务可以访问该资源。是不是很熟悉，在我们的二值信号量里面也是一样的，用于保护临界资源，保证多任务的访问井然有序。当任务获取到信号量的时候才能开始使用被保护的资源，使用完就释放信号量，下一个任务才能获取到信号量从而可用使用被保护的资源。但是信号量会导致的另一个潜在问题，那就是任务**优先级翻转** 。 而 FreeRTOS 提供的互斥量可以通过**优先级继承算法**， 可以降低优先级翻转问题产生的影响，所以，用于临界资源的保护一般建议使用互斥量。

1、优先级继承

优先级继承算法是指，暂时提高某个占有某种资源的低优先级任务的优先级，使之与在所有等待该资源的任务中优先级最高那个任务的优先级相等，而当这个低优先级任务执行完毕释放该资源时，优先级重新回到初始设定值。因此，继承优先级的任务避免了系统资源被任何中间优先级的任务抢占。

互斥量与二值信号量最大的不同是：**互斥量具有优先级继承机制**，而信号量没有。也就是说，某个临界资源受到一个互斥量保护，如果这个资源正在被一个低优先级任务使用，那么此时的互斥量是闭锁状态，也代表了没有任务能申请到这个互斥量，如果此时一个高优先级任务想要对这个资源进行访问，去申请这个互斥量，那么**高优先级任务会因为申请不到互斥量而进入阻塞态**，那么系统会将现在持有该互斥量的任务的优先级临时提升到与高优先级任务的优先级相同，这个优先级提升的过程叫做优先级继承。这个优先级继承机制确保高优先级任务进入阻塞状态的时间尽可能短，以及将已经出现的“**优先级翻转**”危害降低到最小。

为什么说优先级翻转在操作系统中是危害很大？因为在我们一开始创造这个系统的时候，我们就已经设置好了任务的优先级了，越重要的任务优先级越高。但是发生优先级翻转，对我们操作系统是致命的危害，会导致系统的高优先级任务阻塞时间过长。

举个例子，现在有 3 个任务分别为 H 任务（High）、 M 任务（Middle）、 L 任务（Low）， 3 个任务的优先级顺序为 H 任务>M 任务>L 任务。正常运行的时候 H 任务可以打断 M 任务与 L 任务，M 任务可以打断 L 任务，假设系统中有一个资源被保护了，此时该资源被 L 任务正在使用中，某一刻， H 任务需要使用该资源，但是 L 任务还没使用完， H任务则因为申请不到资源而进入阻塞态， L 任务继续使用该资源，此时已经出现了“优先级翻转”现象，高优先级任务在等着低优先级的任务执行，如果在 L 任务执行的时候刚好M 任务被唤醒了，由于 M 任务优先级比 L 任务优先级高，那么会打断 L 任务，抢占了CPU 的使用权，直到 M 任务执行完，再把 CUP 使用权归还给 L 任务， L 任务继续执行，等到执行完毕之后释放该资源， H 任务此时才从阻塞态解除，使用该资源。这个过程，本来

是最高优先级的 H 任务，在等待了更低优先级的 L 任务与 M 任务，其阻塞的时间是 M任务运行时间+L 任务运行时间，这只是只有 3 个任务的系统，假如很多个这样子的任务打断最低优先级的任务，那这个系统最高优先级任务岂不是崩溃了，这个现象是绝对不允许出现的，高优先级的任务必须能及时响应。

2、互斥量应用场景

互斥量的使用比较单一，因为它是信号量的一种，并且它是以锁的形式存在。在初始化的时候，互斥量处于开锁的状态，而被任务持有的时候则立刻转为闭锁的状态。互斥量更适合于：

- 可能会引起优先级翻转的情况。

递归互斥量更适用于：

- 任务可能会多次获取互斥量的情况下。这样可以避免同一任务多次递归持有而造成死锁的问题

多任务环境下往往存在多个任务竞争同一临界资源的应用场景，互斥量可被用于对临界资源的保护从而实现独占式访问。另外，互斥量可以降低信号量存在的优先级翻转问题带来的影响。

比如有两个任务需要对串口进行发送数据，其硬件资源只有一个，那么两个任务肯定不能同时发送啦，不然导致数据错误，那么，就可以用互斥量对串口资源进行保护，当一个任务正在使用串口的时候，另一个任务则无法使用串口，等到任务使用串口完毕之后，另外一个任务才能获得串口的使用权。

另外需要注意的是 互斥量不能在中断服务函数中使用 ，因为其特有的**优先级继承机制只在任务起作用，在中断的上下文环境毫无意义。**

3、互斥量运作机制

多任务环境下会存在多个任务访问同一临界资源的场景，该资源会被任务独占处理。其他任务在资源被占用的情况下不允许对该临界资源进行访问，这个时候就需要用到FreeRTOS 的互斥量来进行资源保护，那么互斥量是怎样来避免这种冲突？用互斥量处理不同任务对临界资源的同步访问时， 任务想要获得互斥量才能进行资源访问，如果一旦有任务成功获得了互斥量，则互斥量立即变为闭锁状态，此时其他任务会因为获取不到互斥量而不能访问这个资源， 任务会根据用户自定义的等待时间进行等待，直到互斥量被持有的任务释放后，其他任务才能获取互斥量从而得以访问该临界资源，此时互斥量再次上锁，如此一来就可以确保每个时刻只有一个任务正在访问这个临界资源，保证了临界资源操作的安全性。

4、常用API

> **递归互斥量创建函数 xSemaphoreCreateRecursiveMutex()**
>
> **互斥量删除函数 vSemaphoreDelete()**
>
> **互斥量获取函数 xSemaphoreTake()**
>
> **递归互斥量获取函数 xSemaphoreTakeRecursive()**
>
> **互斥量释放函数 xSemaphoreGive() 互斥量的释放只能在任务中， 不允许在中断中释放互斥量。**
>
> **递归互斥量释放函数 xSemaphoreGiveRecursive()**

十三、事件

事件是一种实现任务间通信的机制，主要用于实现多任务间的同步，但事件通信只能是事件类型的通信，无数据传输。与信号量不同的是，它可以实现一对多，多对多的同步。即一个任务可以等待多个事件的发生：可以是任意一个事件发生时唤醒任务进行事件处理；也可以是几个事件都发生后才唤醒任务进行事件处理。同样，也可以是多个任务同步多个事件。

在 STM32 中， 我 们 一 般 将configUSE_16_BIT_TICKS 定义为 0，那么 uxEventBits 是 32 位的，有 24 个位用来实现事件标志组。 每一位代表一个事件， 任务通过“逻辑与”或“逻辑或”与一个或多个事件建立关联，形成一个事件组。

事件的“**逻辑或**”也被称作是独立型同步，指的是任务感兴趣的所有事件任一件发生即可被唤醒；

事件“**逻辑与**” 则被称为是关联型同步，指的是任务感兴趣的若干事件都发生时才被唤醒，并且事件发生的时间可以不同步。

任务可以通过设置事件位来实现事件的触发和等待操作。 FreeRTOS 的事件仅用于同步，不提供数据传输功能。

FreeRTOS 提供的事件具有如下特点：

- 事件只与任务相关联，事件相互独立，一个 32 位的事件集合（EventBits_t 类型的变量， 实际可用与表示事件的只有 24 位），用于标识该任务发生的事件类型，其中每一位表示一种事件类型（0 表示该事件类型未发生、 1 表示该事件类型已经发生），一共 24 种事件类型。
- 事件仅用于同步，不提供数据传输功能。
- 事件无排队性，即多次向任务设置同一事件(如果任务还未来得及读走)， 等效于只设置一次。
- 允许多个任务对同一事件进行读写操作。
- 支持事件等待超时机制。

在 FreeRTOS 事件中， 每个事件获取的时候，用户可以选择感兴趣的事件，并且选择读取事件信息标记，它有三个属性，分别是**逻辑与**，**逻辑或**以及**是否清除标记**。当任务等待事件同步时，可以通过任务感兴趣的事件位和事件信息标记来判断当前接收的事件是否满足要求，如果满足则说明任务等待到对应的事件，系统将唤醒等待的任务；否则，任务会根据用户指定的阻塞超时时间继续等待下去。

1、事件的使用场景

FreeRTOS 的事件用于事件类型的通讯，无数据传输，也就是说，我们可以用事件来做标志位（避免了全局变量无法进行有效保护的风险）.

事件可使用于多种场合，它能够在一定程度上替代信号量，用于任务与任务间，中断与任务间的同步。一个任务或中断服务例程发送一个事件给事件对象，而后等待的任务被唤醒并对相应的事件进行处理。但是它与信号量不同的是，事件的发送操作是不可累计的，而信号量的释放动作是可累计的。事件另外一个特性是，接收任务可等待多种事件，即多个事件对应一个任务或多个任务。同时按照任务等待的参数，可选择是“逻辑或”触发还是“逻辑与”触发。这个特性也是信号量等所不具备的，信号量只能识别单一同步动作，而不能同时等待多个事件的同步。

各个事件可分别发送或一起发送给事件对象，而任务可以等待多个事件， 任务仅对感兴趣的事件进行关注。当有它们感兴趣的事件发生时并且符合感兴趣的条件， 任务将被唤醒并进行后续的处理动作。

2、事件运作机制

接收事件时，可以根据感兴趣的参事件类型接收事件的单个或者多个事件类型。事件

接收成功后，必须使用 xClearOnExit 选项来清除已接收到的事件类型，否则不会清除已接收到的事件，这就需要用户显式清除事件位。 用户可以自定义通过传入参数xWaitForAllBits 选择读取模式，是等待所有感兴趣的事件还是等待感兴趣的任意一个事件。

设置事件时，对指定事件写入指定的事件类型，设置事件集合的对应事件位为 1，可以一次同时写多个事件类型， 设置事件成功可能会触发任务调度。

清除事件时，根据入参数事件句柄和待清除的事件类型，对事件对应位进行清 0 操作。

事件不与任务相关联，事件相互独立，一个 32 位的变量（事件集合，实际用于表示事件的只有 24 位），用于标识该任务发生的事件类型，其中每一位表示一种事件类型（0 表示该事件类型未发生、 1 表示该事件类型已经发生）。

事件唤醒机制，当任务因为等待某个或者多个事件发生而进入阻塞态，当事件发生的时候会被唤醒。

3、常用api

> **事件创建函数 xEventGroupCreate()**
>
> **事件删除函数 vEventGroupDelete()**
>
> **事件组置位函数 xEventGroupSetBits(EventGroupHandle_t xEventGroup, const EventBits_t uxBitsToSet); （任务）**
>
> **事件组置位函数 xEventGroupSetBitsFromISR()（中断）**
>
> **等待事件函数 xEventGroupWaitBits() 获取事件是否已经发生**
>
> **清除事件组指定的位**
>
> **xEventGroupClearBits(EventGroupHandle_t xEventGroup, const EventBits_t uxBitsToClear)**
>
> **xEventGroupClearBitsFromISR(EventGroupHandle_t xEventGroup, const EventBits_t uxBitsToClear)**

5、实验

当Key1 和 Key2 都按了（那个先按无所谓，至少按一次），LED灯点亮

```
#include "stm32f10x.h"                  // Device header
#include "Delay.h"
#include "OLED.h"
#include "stdio.h"
#include "LED.h"
#include "KEY.h"
#include "Serial2.h"
#include "FreeRTOS.h"
#include "task.h"
#include "event_groups.h"


//需求： 当Key1 和 Key2 都按了（那个先按无所谓，至少按一次），LED灯点亮
```

```
TaskHandle_t Task_Key1Handle;
TaskHandle_t Task_Key2Handle;
TaskHandle_t Task_LEDHandle;


TaskHandle_t CreateTaskHandle;//创建任务的句柄


 EventGroupHandle_t xCreatedEventGroup;//创建一个事件组的句柄


void Task_Key1Function(void * pvParameters)
{
    while(1)
    {
        if(Key1_Press()==1)
        {
            //给事件组
            xEventGroupSetBits(xCreatedEventGroup,0x01); //KEY1的bit位下标为0
        }


    }
}


void Task_Key2Function(void * pvParameters)
{
    while(1)
    {
        if(Key2_Press()==1)
        {
            //给事件组
            xEventGroupSetBits(xCreatedEventGroup,0x02); //KEY2的bit位下标为1
        }


    }
}


void Task_LED(void * pvParameters)
{

    while(1)
    {
        //当KEY1和KEY2都按下，才点亮灯
        EventBits_t Bits = xEventGroupWaitBits(xCreatedEventGroup,
                           3,
                           pdTRUE, //等待成功，pdFALSE不清除,pdTRUE清除
```

```
                                pdTRUE,//参数传递的值中的所有位在事件组中被设置 或指定的阻塞时间到
期
                                10000 //等待以下情况发生的最长时间（以“滴答”为单位） -- 10000 就是
10秒
        );//1.超时会返回Bits， 如果等待成功了会提前返回Bits

        if(Bits==3){
            Serial2_SendString("Both KEY1 and KEY2 Press. \r\n");
            LED1_ON();
        }else if(Bits==1)
        {
            Serial2_SendString("Only KEY1  Press. \r\n");
        }else if(Bits ==2)
        {
            Serial2_SendString("Only KEY2  Press. \r\n");
        }else{
            Serial2_SendString("Not KEY  Press. \r\n");
        }


    }
}



void CreateTask(void * pvParameters)
{
    taskENTER_CRITICAL();
    xTaskCreate(Task_Key1Function,"Task_Key1Function",128,NULL,1,&Task_Key1Handle);
    xTaskCreate(Task_Key2Function,"Task_Key2Function",128,NULL,1,&Task_Key2Handle);
    xTaskCreate(Task_LED,"Task_LED",128,NULL,1,&Task_LEDHandle);

    taskEXIT_CRITICAL();
    vTaskDelete( NULL );//删除自己，别忘记
}



int main(void)
{
    //初始化硬件
    OLED_Init();
    OLED_Clear();
    LED_Init();

    KEY_Init();

    Serial2_Init();

    //创建一个事件组
    xCreatedEventGroup = xEventGroupCreate();
```

```
    //创建任务
    xTaskCreate(CreateTask,"CreateTask",128,NULL,3,&CreateTaskHandle);


    //把任务跑起来。
    vTaskStartScheduler();
    //后面的代码执行不到了
}
```

十四、软件定时器

使用软件定时器时，需要我们在创建软件定时器时指定时间到达后要调用的函数（也称超时函数/回调函数），在回调函数中处理信息。

FreeRTOS 操作系统提供软件定时器功能，软件定时器的使用相当于扩展了定时器的数量，允许创建更多的定时业务。 FreeRTOS 软件定时器功能上支持：

- 软件定时器创建。
- 软件定时器启动。
- 软件定时器停止。
- 软件定时器复位。
- 软件定时器删除

FreeRTOS 提供的软件定时器支持单次模式和周期模式

单次模式：当用户创建了定时器并启动了定时器后，定时时间到了，只执行一次回调函数之后就将该定时器进入休眠状态，不再重新执行。

周期模式：这个定时器会按照设置的定时时间循环执行回调函数，直到用户将定时器删除。

FreeRTOS 通过一个 prvTimerTask 任务（也叫守护任务 Daemon）管理软定时器，它是在启动调度器时自动创建的， 为了满足用户定时需求。 prvTimerTask 任务会在其执行期间检查用户启动的时间周期溢出的定时器，并调用其回调函数。

软件定时器以系统节拍周期为计时单位。

软件定时器是可选的系统资源，在创建定时器的时候会分配一块内存空间。

当用户创建并启动一个软件定时器时， FreeRTOS 会根据当前系统时间及用户设置的定时确定该定时器唤醒时间，并将该定时器控制块挂入**软件定时器列表**， FreeRTOS 中采用**两个定时器列表维护软件定时器**， **pxCurrentTimerList** 与 **pxOverflowTimerList** 是列表指针， 在初始化的时候分别指向 **xActiveTimerList1** 与 **xActiveTimerList2**

pxCurrentTimerList： 系统新创建并激活的定时器都会以**超时时间升序**的方式插入到 pxCurrentTimerList 列表中。系统在定时器任务中扫描 pxCurrentTimerList 中的第一个定时器，看是否已超时，若已经超时了则调用软件定时器回调函数。否则将定时器任务挂起，因为定时时间是升序插入软件定时器列表的，列表中第一个定时器的定时时间都还没到的话，那后面的定时器定时时间自然没到。

pxOverflowTimerList 列表是在软件定时器溢出的时候使用， 作用与 pxCurrentTimerList一致。

随着 SysTick 的触发一直在增长（每一次硬件定时器中断来临， xTimeNow 变量会加 1），在软件定时器任务运行的时候会获取下一个要唤醒的定时器，比较当前系统时间xTimeNow 是否大于或等于下一个定时器唤醒时间 xTicksToWait，若大于则表示已经超时，定时器任务将会调用对应定时器的回调函数，否则将软件定时器任务挂起，直至下一个要唤醒的软件定时器时间到来或者接收到命令消息。

使用软件定时器时候要注意以下几点：

- 软件定时器的回调函数中应快进快出，绝对不允许使用任何可能引软件定时器起任务挂起或者阻塞的 API 接口，在回调函数中也绝对不允许出现死循环。
- 软件定时器使用了系统的一个队列和一个任务资源，软件定时器任务的优先级默认为 configTIMER_TASK_PRIORITY，为了更好响应，该优先级应设置为所有任务中最高的优先级。创建单次软件定时器，该定时器超时执行完回调函数后，系统会自动删除该软件定时器，并回收资源。
- 定时器任务的堆栈大小默认为 configTIMER_TASK_STACK_DEPTH 个字节。

1、常用api

> **软件定时器创建函数　xTimerCreate()**
>
> **软件定时器启动函数　xTimerStart()　xTimerStartFromISR()**
>
> **软件定时器停止函数　xTimerStop()　xTimerStopFromISR()**
>
> **软件定时器删除函数　xTimerDelete()**

2、实验

在按下key后LED亮，10秒内没有再次按下key，LED灭。

```
#include "stm32f10x.h"                  // Device header
#include "Delay.h"
#include "OLED.h"
#include "stdio.h"
#include "LED.h"
#include "KEY.h"
#include "Serial2.h"
#include "FreeRTOS.h"
#include "task.h"
#include "event_groups.h"
#include "timers.h"


//需求：  在按下key后LED亮，10秒内没有再次按下key，LED灭。

TaskHandle_t Task_Key1Handle;
TaskHandle_t Task_LEDHandle;


TaskHandle_t CreateTaskHandle;//创建任务的句柄

TimerHandle_t MyTimerHandler;
```

```
void Task_Key1Function(void * pvParameters)
{
    TickType_t xRemainingTime;
    char str[200];
    while(1)
    {
        if(Key1_Press()==1)
        {
            //给事件组
            LED1_ON();
            xTimerStart(MyTimerHandler,0);//启动


        }
        xRemainingTime = xTimerGetExpiryTime( MyTimerHandler ) - xTaskGetTickCount();
        sprintf(str,"%d\n",xRemainingTime);
        Serial2_SendString(str);


    }
}




void CreateTask(void * pvParameters)
{
    taskENTER_CRITICAL();
    xTaskCreate(Task_Key1Function,"Task_Key1Function",128,NULL,1,&Task_Key1Handle);
    taskEXIT_CRITICAL();
    vTaskDelete( NULL );//删除自己，别忘记
}

void vCallbackFunction( TimerHandle_t xTimer )
{
    LED1_OFF();
}

int main(void)
{
    //初始化硬件
    OLED_Init();
    OLED_Clear();

    LED_Init();

    KEY_Init();
```

```
    Serial2_Init();

    //创建一个事件组
     MyTimerHandler = xTimerCreate
    (               "LEDON_Timer",//定时器名字
                    5000,//定时器的周期
                    pdFAIL,//如果 uxAutoReload 设置为 pdTRUE， 那么定时器将以 xTimerPeriod 参数设置的频率重复过期。如果 uxAutoReload 设置为 pdFALSE，则此定时器为一次性定时器， 它会在到期后进入休眠状态
                    0,//分配给正在创建的定时器的标识符
                    vCallbackFunction  //定时器到期时调用的函数
    );


    //创建任务
    xTaskCreate(CreateTask,"CreateTask",128,NULL,3,&CreateTaskHandle);


    //把任务跑起来。
    vTaskStartScheduler();
    //后面的代码执行不到了


}
```

十五、任务通知

FreeRTOS 从 V8.2.0 版本开始提供任务通知这个功能，每个任务都有一个 32 位的通知值，在大多数情况下，任务通知可以替代二值信号量、计数信号量、事件组， 也可以替代长度为 1 的队列（可以保存一个 32 位整数或指针值） 。

FreeRTOS 提供以下几种方式发送通知给任务 ：

- 发送通知给任务， 如果有通知未读，不覆盖通知值。
- 发送通知给任务，直接覆盖通知值。
- 发送通知给任务，设置通知值的一个或者多个位，可以当做事件组来使用。
- 发送通知给任务，递增通知值，可以当做计数信号量使用。

限制 ：

- 只能有一个任务接收通知消息， 因为必须指定接收通知的任务
- 只有等待通知的任务可以被阻塞， 发送通知的任务，在任何情况下都不会因为发送失败而进入阻塞态

任务通知是属于任务中附带的资源， 所以在任务被创建的时候，任务通知也被初始化的。由于任务通知的数据结构包含在任务控制块中，只要任务存在，任务通知数据结构就已经创建完毕，可以直接使用，所以使用的时候很是方便。

1、常用api

发送通知 API 函数，有六个，能完成 3 种操作，每种操作有两个 API 函数，分别为带中断保护版本和不带中断保护版本。

发送任务通知函数　xTaskGenericNotify()

xTaskNotifyGive() 向一个任务发送通知，并将对方的任务通知值加 1

vTaskNotifyGiveFromISR() 用于在中断中向指定任务发送任务通知，并更新对方的任务通知值（加 1 操作）

xTaskNotify() 用于在任务中直接向另外一个任务发送一个事件， 接收到该任务通知的任务有可能解锁。 如果你想使用任务通知来实现二值信号量和计数信号量，那么应该使用更加简单的函数 xTaskNotifyGive()， 而不是使用 xTaskNotify()， xTaskNotify()函数在发送任务通知的时候会指定一个通知值， 并且用户可以指定通知值发送的方式。

xTaskNotifyFromISR() 是xTaskNotify()的中断保护版本

xTaskGenericNotifyFromISR() 中断中发送任务通知通用函数

xTaskNotifyAndQuery() 与 xTaskNotify() 很像， 都是调用通用的任务通知发送函数 xTaskGenericNotify()来实现通知的发送,多了一个附加的参数pulPreviousNotifyValue 用于回传接收任务的上一个通知值

xTaskNotifyAndQueryFromISR() 是 xTaskNotifyAndQuery ()的中断版本

获取任务通知函数(获取任务通知函数只能用在任务中，没有带中断保护版本**)**

ulTaskNotifyTake() 作为二值信号量和计数信号量的一种轻量级实现，速度更快。 如果 FreeRTOS 中使用函数 xSemaphoreTake() 来获取信号量， 这个时候则可以试试使用函数 ulTaskNotifyTake()来代替

xTaskNotifyWait() 全功能版的等待任务通知

十六、内存管理

heap_1.c

heap_1.c 管理方案是 FreeRTOS 提供所有内存管理方案中最简单的一个，它只能申请内存而不能进行内存释放，并且申请内存的时间是一个常量，这样对于要求安全的嵌入式设备来说是最好的，因为不允许内存释放，就不会产生内存碎片而导致系统崩溃，但是也有缺点，那就是内存利用率不高，某段内存只能用于内存申请的地方，即使该内存只使用一次，也无法让系统回收重新利用。

heap1.c 方案具有以下特点：

1. 用于从不删除任务、队列、信号量、互斥量等的应用程序（实际上大多数使用FreeRTOS 的应用程序都符合这个条件）。
2. 函数的执行时间是确定的并且不会产生内存碎片.

heap_2.c

heap_2.c 采用一种最佳匹配算法

> 比如我们申请 100 字节的内存，而可申请内存中有三块对应大小 200 字节， 500 字节和 1000 字节大小的内存块，按照算法的最佳匹配，这时候系统会把 200 字节大小的内存块进行分割并返回申请内存的起始地址，剩余的内存则插回链表留待下次申请

heap_2.c 方案具有以下特点：

1. 可以用在那些反复的删除任务、队列、信号量、等内核对象且不担心内存碎片的应用程序。
2. 如果我们的应用程序中的队列、任务、信号量、 等工作在一个不可预料的顺序，这样子也有可能会导致内存碎片。
3. 具有不确定性，但是效率比标准 C 库中的 malloc 函数高得多
4. 不能用于那些内存分配和释放是随机大小的应用程序。

heap_3.c

heap_3.c 方案只是简单的封装了标准 C 库中的 malloc()和 free()函数， 并且能满足常用的编译器。 重新封装后的 malloc()和 free()函数具有保护功能，采用的封装方式是操作内存前挂起调度器、完成后再恢复调度器。

heap_3.c 方案具有以下特点：

1. 需要链接器设置一个堆， malloc()和 free()函数由编译器提供。
2. 具有不确定性。
3. 很可能增大 RTOS 内核的代码大小。

heap_4.c

heap_4.c 方案与 heap_2.c 方案一样都采用最佳匹配算法来实现动态的内存分配，但是不一样的是 heap_4.c 方案还包含了一种合并算法，能把相邻的空闲的内存块合并成一个更大的块，这样可以减少内存碎片。

heap_5.c

heap_5.c 方案在实现动态内存分配时与 heap4.c 方案一样， 采用最佳匹配算法和合并算法，并且允许内存堆跨越多个非连续的内存区，也就是允许在不连续的内存堆中实现内存分配，比如用户在片内 RAM 中定义一个内存堆，还可以在外部 SDRAM 再定义一个或多个内存堆，这些内存都归系统管理。

总结

文件	优点	缺点
heap_1	分配简单，时间确定	只能申请，不能释放
heap_2	可以动态分配	不能合并相邻内存，产生内存碎片
heap_3	对标准C库的malloc、free进行封装，提供了线程保护。	速度慢，时间不定
heap_4	动态分配，可以合并相信空闲内存(常用)	时间不定
heap_5	在heap4基础上能管理多个非连续的内存	时间不定

> 大部分嵌入式设备内存相对简单，通常是单一连续的内存区域。 `heap_4` 作为更早出现的实现，在实际项目中被广泛测试，已被证明足够稳定。易于配置、效率高且功能满足绝大多数需求。

> 对于绝大多数使用场景， `heap_4` 提供的功能和性能已经能够满足需求，开发者倾向于选择它而不是使用更复杂的 `heap_5` 。